

**BỘ CÔNG THƯƠNG
TRƯỜNG ĐẠI HỌC CÔNG THƯƠNG
THÀNH PHỐ HỒ CHÍ MINH**



BÁO CÁO TỔNG KẾT ĐỀ TÀI

**XÂY DỰNG MÔ HÌNH NHẬN DIỆN ĐỘNG VẬT TRONG SÁCH
ĐỎ VIỆT NAM**

Học phần: Deep Learning

Mã lớp: 010110195612

Nhóm thực hiện: 9

- 1. Đặng Thành Thi – 2001230918 (Nhóm trưởng)**
- 2. Lưu Đức Linh – 2001230444**
- 3. Trần Xuân Hương – 2001230339**

GVHD: Huỳnh Thái Học

THÀNH PHỐ HỒ CHÍ MINH – NĂM 2026

MỤC LỤC

LỜI MỞ ĐẦU	4
CHƯƠNG 1: GIỚI THIỆU ĐỀ TÀI.....	6
1.1 Bối cảnh và động cơ thực hiện.....	6
1.2 Nhu cầu thực tiễn và bài toán cần giải quyết.....	8
1.3 Mục tiêu nghiên cứu.....	10
1.4 Phạm vi và giới hạn đề tài.....	12
1.5 Công nghệ và ngôn ngữ lập trình chính.....	14
CHƯƠNG 2: KIẾN TRÚC HỆ THỐNG VÀ CÁC PHÂN HỆ CHÍNH.....	16
2.1 Sơ đồ luồng hoạt động hệ thống.....	16
2.2 Phân hệ Kỹ thuật Dữ liệu.....	21
2.3 Phân hệ Huấn luyện & Đánh giá mô hình AI (Model Pipeline).....	23
2.4 Phân hệ Phát triển Giao diện Ứng dụng.....	26
CHƯƠNG 3: TẬP DỮ LIỆU & TIỀN XỬ LÝ.....	29
3.1 Giới thiệu Dataset 9 loài động vật hoang dã.....	29
3.2 Kỹ thuật Augmentation và Cân bằng lớp dữ liệu.....	32
3.3 Phân chia tập dữ liệu huấn luyện khoa học.....	34
CHƯƠNG 4: THIẾT KẾ GIAO DIỆN & TRỰC QUAN HÓA MÔ HÌNH AI.....	39
4.1 Thiết kế ứng dụng Desktop chuyên nghiệp.....	39
4.2 Nguyên lý trực quan hóa Explainable AI.....	43
4.3 Thiết kế ứng dụng Web trực tuyến.....	48
CHƯƠNG 5: VẬN DỤNG LÝ THUYẾT DEEP LEARNING VÀO DỰ ÁN.....	52
5.1 Kiến trúc ResNet-18 Transfer Learning và Cơ chế Đóng băng tham số.....	52
5.2 Hàm mất mát CrossEntropyLoss với Label Smoothing và Bộ tối ưu hóa Adam.....	55
5.3 Phân tích kết quả thực nghiệm và các biểu đồ hiệu năng.....	58

LỜI KẾT	65
TÀI LIỆU THAM KHẢO	68

LỜI MỞ ĐẦU

Trong xu thế phát triển mạnh mẽ của khoa học công nghệ toàn cầu cùng với sự gia tăng các thách thức về môi trường sinh thái, bảo tồn đa dạng sinh học đã trở thành một trong những ưu tiên hàng đầu mang tính sống còn đối với nhân loại nhằm duy trì sự cân bằng sinh thái trên Trái Đất. Việt Nam, với đặc trưng địa lý trải dài trên nhiều vĩ độ và sở hữu các hệ sinh thái rừng rậm nhiệt đới phong phú, được công nhận là một trong những trung tâm đa dạng sinh học lớn nhất thế giới. Tuy nhiên, quốc gia cũng đang đứng trước áp lực không nhỏ từ việc bảo tồn các loài động vật hoang dã nguy cấp, quý hiếm. Sự suy giảm nhanh chóng số lượng cá thể của các loài đặc hữu mang tính biểu tượng như Voọc Chà Vá, Tê Tê Java, Cu Li Nhỏ hay Hổ Đông Dương đòi hỏi phải có những biện pháp giám sát sinh thái chặt chẽ, liên tục và chính xác. Thực tế cho thấy, việc giám sát sinh cảnh thủ công truyền thống thông qua các chuyến tuần tra thực địa của kiểm lâm hoặc phân tích thủ công hình ảnh từ hệ thống bẫy ảnh bằng mắt thường đang bộc lộ những hạn chế không thể phủ nhận: chi phí vận hành lớn, tốn kém nguồn nhân lực mỏng và độ chính xác bị ảnh hưởng nhiều bởi các yếu tố chủ quan của con người cũng như điều kiện thời tiết phức tạp tại các vùng rừng sâu rậm rạp.

Thực tế hiện nay cho thấy, sự suy giảm nghiêm trọng của hệ sinh thái tự nhiên và nạn săn bắt động vật hoang dã trái phép đang diễn ra ở mức báo động trên toàn cầu, đặc biệt là tại các quốc gia vùng nhiệt đới giàu đa dạng sinh học như Việt Nam. Theo các báo cáo của Tổ chức Bảo tồn Thiên nhiên Quốc tế (IUCN), tốc độ tuyệt chủng của các loài động vật hoang dã trong thế kỷ qua đã nhanh gấp hàng trăm lần so với tốc độ tuyệt chủng tự nhiên trong lịch sử Trái Đất. Trong bối cảnh đó, các phương pháp tuần tra rừng và giám sát động vật thủ công truyền thống của lực lượng kiểm lâm không còn đáp ứng kịp tốc độ tàn phá của các đường dây săn bắt trộm hay sự biến đổi nhanh chóng của sinh cảnh. Sự xuất hiện của công nghệ giám sát tự động bằng bẫy ảnh đã mở ra một hướng đi mang tính cách mạng, cho phép thu thập hàng triệu bức ảnh về thế giới động vật trong rừng sâu một cách liên tục mà không làm xáo trộn đời sống tự nhiên của chúng.

Sự trỗi dậy mang tính đột phá của Trí tuệ Nhân tạo, đặc biệt là các công nghệ học sâu Deep Learning và thị giác máy tính, đã mang lại những công cụ đầy triển vọng

giúp giải quyết bài toán giám sát sinh thái ở quy mô lớn. Bằng việc ứng dụng các mạng nơ-ron tích chập (CNN) sâu, hệ thống máy tính có khả năng tự động nhận diện, định danh và phân loại động vật từ dữ liệu hình ảnh hay video với tốc độ xử lý nhanh chóng trong vài mili-giây và độ chính xác tiệm cận con người. Thêm vào đó, sự xuất hiện của kỹ thuật Học chuyển giao đã khắc phục được điểm nghẽn lớn nhất trong việc huấn luyện các mô hình học máy chuyên sâu: sự thiếu hụt dữ liệu hình ảnh thực tế của các loài động vật quý hiếm do chúng rất khó bắt gặp trong tự nhiên. Bằng cách kế thừa các tri thức trích xuất đặc trưng hình học sẵn có từ các mô hình đã được huấn luyện trước trên hàng triệu hình ảnh tổng quát của tập dữ liệu ImageNet, chúng ta có thể huấn luyện hiệu quả các mô hình học sâu trên các tập dữ liệu nhỏ của động vật hoang dã mà vẫn đạt được độ chính xác phân loại tối ưu, giảm thiểu đáng kể chi phí tính toán và thời gian huấn luyện.

Mặc dù bấy ảnh giải quyết được bài toán thu thập dữ liệu sinh thái ở quy mô lớn, nhưng nó lại trực tiếp tạo ra một điểm nghẽn nghiêm trọng khác cho công tác nghiên cứu: sự quá tải dữ liệu (data overload). Một đợt đặt bấy ảnh kéo dài vài tháng tại một khu bảo tồn hay vườn quốc gia có thể thu về từ hàng trăm ngàn đến hàng triệu bức ảnh tĩnh và video ngắn. Việc lọc sạch và phân loại thủ công lượng dữ liệu khổng lồ này đòi hỏi hàng ngàn giờ làm việc căng thẳng của các chuyên gia sinh học và kiểm lâm viên, dẫn đến độ trễ lớn trong việc đưa ra các quyết định hành động bảo tồn kịp thời. Do đó, nhu cầu cấp thiết hiện nay là phát triển một giải pháp tự động hóa thông minh có khả năng nhận dạng loài một cách chính xác từ dữ liệu bấy ảnh thô, giúp nhanh chóng lọc bỏ các ảnh không chứa đối tượng quan tâm (như ảnh nhiễu do gió thổi cành cây hoặc thay đổi ánh sáng đột ngột) và phân loại chi tiết các loài quý hiếm.

Đề tài “Xây dựng hệ thống nhận diện 9 loài động vật Sách Đỏ Việt Nam dựa trên kiến trúc ResNet-18 Transfer Learning tích hợp giải thích mô hình Grad-CAM” được thực hiện nhằm mục đích thiết lập một quy trình khép kín chuẩn khoa học học máy từ khâu thu thập ảnh thô, tăng cường dữ liệu cân bằng lớp, huấn luyện mô hình sâu, đến đánh giá định lượng khoa học và đóng gói ứng dụng thực tế. Điểm đặc biệt của đề tài này là việc tích hợp thành công thuật toán Grad-CAM (Gradient-weighted Class Activation Mapping). Công cụ Explainable AI này giúp trực quan hóa vùng tập trung chú ý của AI dưới dạng bản đồ nhiệt, giải quyết triệt để vấn đề hộp đen của Deep Learning, tạo

ra sự minh bạch, tăng mức độ tin cậy khoa học đối với các kiểm lâm viên và nhà quản lý bảo tồn khi ứng dụng phần mềm vào thực tế dã ngoại. Báo cáo này trình bày chi tiết toàn bộ cơ sở toán học, thiết kế hệ thống, các kết quả thực nghiệm chi tiết và các thảo luận khoa học nghiêm túc của nhóm nghiên cứu đối với học phần Deep Learning.

Đề tài này không chỉ dừng lại ở việc thử nghiệm các mô hình học sâu trên máy tính cá nhân, mà hướng tới việc xây dựng một hệ thống hoàn chỉnh từ khâu thu thập, tiền xử lý dữ liệu sạch, huấn luyện mô hình tối ưu cho đến việc triển khai các ứng dụng thực tế trên môi trường Desktop và Web để kiểm lâm viên hay nhà nghiên cứu sinh thái có thể sử dụng trực tiếp tại thực địa. Sự kết hợp giữa lý thuyết Deep Learning hiện đại và các giải pháp lập trình ứng dụng thực tế sẽ tạo nên một công cụ hữu ích, đóng góp thiết thực cho công cuộc bảo vệ tài nguyên thiên nhiên quốc gia bằng công nghệ số hóa.

CHƯƠNG 1: GIỚI THIỆU ĐỀ TÀI

1.1 Bối cảnh và động cơ thực hiện

Việt Nam sở hữu hệ sinh thái rừng nhiệt đới phong phú, là địa bàn phân bố tự nhiên của nhiều loài động vật quý hiếm. Tuy nhiên, do ảnh hưởng tiêu cực từ nạn phá rừng và nạn săn bắt động vật hoang dã trái phép, nhiều loài đặc hữu hiện đang đối mặt với nguy cơ tuyệt chủng. Theo các báo cáo chính thức từ IUCN và WWF, số lượng cá thể ngoài tự nhiên của nhiều loài tại nước ta đã suy giảm ở mức báo động đỏ: Hổ Đông Dương (*Panthera tigris corbetti*) hiện ước tính còn dưới 5 cá thể ngoài tự nhiên; Hươu Sao (*Cervus nippon*) đã tuyệt chủng ngoài tự nhiên từ năm 2002; Voi Châu Á (*Elephas maximus*) chỉ còn dưới 100 cá thể phân bố rải rác trong các quần thể bị chia cắt; và Voọc Chà Vá Chân Nâu (*Pygathrix nemaeus*) đang ở mức cực kỳ nguy cấp (CR) do mất môi trường sống tại các khu rừng đặc dụng miền Trung. Do đó, việc nghiên cứu các giải pháp giám sát, nhận diện nhanh các loài động vật quý hiếm là vô cùng cấp thiết, hỗ trợ kiểm lâm kịp thời bảo tồn rừng và ngăn chặn nạn săn trộm.

Sự đa dạng sinh học của Việt Nam được thế giới công nhận với nhiều loài đặc hữu quý hiếm có tên trong Sách Đỏ. Tuy nhiên, các loài như Voọc Chà Vá, Cu Li Nhỏ, Vượn Má Vàng hay Tê Tê Java đang đứng trước nguy cơ biến mất hoàn toàn ngoài tự

nhiên do mất sinh cảnh sống và nạn săn bắt trộm quy mô lớn. Việc giám sát liên tục sự hiện diện và mật độ quần thể của các loài này là cơ sở khoa học cốt lõi để các vườn quốc gia đưa ra phương án bảo vệ phù hợp. Phương pháp truyền thống dựa vào việc đi tuần tra tìm kiếm dấu chân, chất thải hoặc quan sát trực tiếp bằng mắt thường thường rất khó khăn, nguy hiểm và có độ chính xác thấp do động vật hoang dã có tập tính trốn tránh con người và hoạt động chủ yếu về đêm.

Để cứu vãn các loài sinh vật này trước khi quá muộn, các ban quản lý rừng đặc dụng và vườn quốc gia tại Việt Nam như Cát Tiên, Bạch Mã hay Cúc Phương đã tích cực số hóa công tác giám sát sinh thái bằng việc thiết lập mạng lưới máy bẫy ảnh tự động dày đặc. Đây là các camera tích hợp cảm biến chuyển động và cảm biến nhiệt hồng ngoại được đặt cố định tại các lối đi sâu của thú rừng nhằm ghi lại hình ảnh tự động khi có sinh vật đi qua. Mặc dù công nghệ này giúp thu thập hàng triệu bức ảnh thực địa vô giá mỗi năm, việc phân tích thủ công lượng dữ liệu khổng lồ này lại trở thành một nút cổ chai nghiêm trọng về mặt nhân lực. Đội ngũ kiểm lâm mỏng và các chuyên gia sinh học phải tiêu tốn hàng ngàn giờ làm việc căng thẳng chỉ để lọc bỏ ảnh trống (do gió thổi lá cây kích hoạt cảm biến) và định danh thủ công tên loài. Điều này làm chậm trễ đáng kể khả năng phát hiện các loài có nguy cơ tuyệt chủng cao cũng như các phản ứng kịp thời trước các mối đe dọa săn trộm.

Công nghệ bẫy ảnh tự động kích hoạt khi có cảm biến hồng ngoại phát hiện chuyển động (passive infrared motion-triggered camera traps) đã được triển khai rộng rãi tại các vườn quốc gia lớn. Các thiết bị này hoạt động liên tục 24/7 trong mọi điều kiện thời tiết khắc nghiệt, chụp lại tất cả những gì di chuyển qua ống kính. Tuy nhiên, hơn 80% số ảnh thu được từ bẫy ảnh thường là ảnh nhiễu (không có động vật mà chỉ có lá cây đung đưa, ánh sáng thay đổi hoặc mưa gió). Việc lọc bỏ đồng ảnh nhiễu này bằng sức người là một sự lãng phí tài nguyên nhân lực vô cùng lớn, cản trở việc triển khai bẫy ảnh ở quy mô rộng lớn hơn và làm giảm tính thời sự của thông tin bảo tồn.

Chính vì vậy, động cơ lớn nhất thúc đẩy nhóm nghiên cứu thực hiện đề tài này là ứng dụng các tiến bộ công nghệ học sâu để tự động hóa hoàn toàn quy trình nhận diện động vật. Sự tích hợp giữa khả năng trích xuất đặc trưng không gian mạnh mẽ của kiến trúc mạng tích chập ResNet-18 và công nghệ giải thích trực quan Grad-CAM sẽ

giải quyết triệt để nút thắt cổ chai về dữ liệu giám sát. Hệ thống giúp rút ngắn thời gian phân tích từ hàng tháng xuống còn vài mili-giây cho mỗi bức ảnh với độ tin cậy vượt trội. Điều này không chỉ hỗ trợ đắc lực cho các kiểm lâm viên trong công tác quản lý bảo tồn thực địa, nâng cao nhận thức bảo vệ động vật hoang dã của cộng đồng, mà còn đóng góp một nghiên cứu ứng dụng công nghệ học sâu có ý nghĩa thực tiễn to lớn đối với học phần Deep Learning của trường.

1.2 Nhu cầu thực tiễn và bài toán cần giải quyết

Để xây dựng một mô hình học sâu có khả năng nhận diện động vật hoạt động chính xác và đáng tin cậy trong môi trường thực địa hoang dã phức tạp, hệ thống nghiên cứu phải đối mặt và giải quyết triệt để ba bài toán kỹ thuật lớn sau:

Bài toán thứ nhất là sự mất cân bằng lớp dữ liệu nghiêm trọng (Class Imbalance). Trong sinh thái học thực địa, tần suất chụp được ảnh của các loài cực kỳ nguy cấp như Tê Tê Java hay Hổ Đông Dương là vô cùng hiếm hoi so với các loài động vật hoang dã phổ thông khác. Sự chênh lệch quá lớn về mặt số lượng mẫu ảnh giữa các lớp sẽ làm sai lệch nghiêm trọng quá trình tối ưu hóa hàm mất mát của mạng nơ-ron tích chập. Mô hình sẽ có xu hướng tối ưu hóa để nhận diện tốt các loài có nhiều dữ liệu (lớp đa số) và gần như bỏ qua hoặc nhận diện sai các loài có rất ít mẫu (lớp thiểu số) - vốn lại là các loài quý hiếm nhất cần bảo tồn. Để giải quyết, nhu cầu thực tiễn đòi hỏi phải nghiên cứu và áp dụng các kỹ thuật tăng cường dữ liệu hình ảnh (Image Augmentation) một cách khoa học để nhân bản dữ liệu một cách tự nhiên cho các lớp thiếu ảnh, đồng thời phải thực hiện phân chia tập dữ liệu huấn luyện, kiểm thử (Train, Validation, Test) độc lập trước khi tăng cường ảnh để ngăn chặn hoàn toàn hiện tượng rò rỉ dữ liệu (Data Leakage) làm sai lệch kết quả đánh giá.

Sự mất cân bằng lớp dữ liệu là một thách thức toán học kinh điển trong học sâu. Khi huấn luyện mô hình với số lượng ảnh quá chênh lệch giữa các lớp (ví dụ, loài phổ biến như Hươu Sao có hàng ngàn ảnh trong khi loài cực kỳ nguy cấp như Báo Gấm chỉ có vài chục ảnh), hàm mất mát Cross-Entropy truyền thống sẽ bị chi phối bởi các lớp đa số. Kết quả là mô hình sẽ có xu hướng dự đoán thiên lệch về lớp đa số để giảm thiểu tổng tổn thất, dẫn đến độ nhạy (Recall) đối với các loài quý hiếm bị sụt giảm nghiêm trọng. Hệ thống cần áp dụng các kỹ thuật cân bằng lớp ở cả mức dữ liệu (tăng

cường ảnh chọn lọc) và mức thuật toán (như Label Smoothing) để đảm bảo mô hình học được các đặc trưng phân biệt bình đẳng cho tất cả các loài.

Bài toán thứ hai là tính minh bạch và khả năng giải thích được của AI (Explainable AI - XAI). Các mạng nơ-ron tích chập sâu thường hoạt động như một hộp đen toán học, nơi các đặc trưng hình ảnh được biến đổi qua hàng triệu tham số phi tuyến tính phức tạp. Người dùng cuối (như kiểm lâm viên) không thể biết được mô hình dựa trên cơ sở đặc trưng nào để phân loại con vật. Trong thực tế bảo tồn rừng nhiệt đới, việc mô hình nhận diện nhầm (ví dụ nhận diện phong nền bụi cỏ thành một con thú do background bias) sẽ gây ra các báo động giả. Hệ thống cần tích hợp thuật toán trực quan hóa như Grad-CAM (Gradient-weighted Class Activation Mapping). Bằng cách tính toán đạo hàm của điểm số dự đoán lớp mục tiêu đối với bản đồ kích hoạt của lớp tích chập cuối cùng, Grad-CAM sẽ tạo ra bản đồ nhiệt (heatmap) chỉ rõ vùng trọng tâm mà AI đang chú ý (như vùng vẫy của Tê Tê, sọc lông của Hổ, dải chữ V trên ngực Gấu Ngựa). Điều này giúp kiểm chứng xem AI có thực sự 'nhìn' vào con vật hay bị đánh lừa bởi phong nền phức tạp của rừng rậm.

Các mô hình mạng nơ-ron tích chập sâu CNN thường bị coi là các hộp đen vì rất khó để con người hiểu được tại sao mô hình lại đưa ra một dự đoán cụ thể. Trong ứng dụng bảo tồn lâm nghiệp, tính minh bạch này cực kỳ quan trọng. Nếu một mô hình phân loại một bức ảnh là 'Hổ Đông Dương' nhưng thực chất nó lại đưa ra quyết định dựa trên các đặc trưng nền cảnh (như màu đất hoặc lá cây khô xung quanh) thay vì các sọc trên thân hổ, mô hình đó sẽ hoạt động sai lệch khi gặp ảnh hổ ở các môi trường khác. Việc tích hợp công nghệ Explainable AI thông qua thuật toán Grad-CAM sẽ giúp trực quan hóa vùng tập trung chú ý của mô hình dưới dạng bản đồ nhiệt, giúp các nhà nghiên cứu kiểm chứng được độ tin cậy và cơ sở khoa học của quyết định phân loại.

Bài toán thứ ba là khả năng phát hiện dữ liệu ngoài phân phối (Out-of-distribution detection). Trong môi trường thực tế, máy bẫy ảnh chụp được rất nhiều ảnh phong cảnh trống (chỉ có lá cây chuyển động do gió) hoặc các loài động vật thông thường (như chuột, chim, gia súc thả rông) không nằm trong danh mục 9 loài động vật Sách Đỏ cần nhận diện. Nếu đưa các bức ảnh này vào mô hình phân loại đa lớp thông

thường, mạng nơ-ron vẫn sẽ cố gắng gán cho nó một nhãn nhị phân trong số 9 loài với tổng xác suất softmax bằng 100%. Điều này dẫn đến các sai số cảnh báo nghiêm trọng. Thực tiễn yêu cầu phải thiết lập một bộ lọc ngưỡng tin cậy (Confidence Threshold Filter). Nếu xác suất dự đoán của lớp cao nhất dưới một ngưỡng an toàn thiết lập (mặc định là 85%), hệ thống sẽ tự động phân loại bức ảnh đó là 'loài lạ', bảo vệ tính chính xác tuyệt đối cho hệ thống cảnh báo.

Trong môi trường tự nhiên, camera traps sẽ chụp được rất nhiều đối tượng nằm ngoài danh sách các loài mục tiêu huấn luyện, chẳng hạn như mèo nhà, chó hoang, gia súc thả rông, con người (khách du lịch, người dân đi rừng, kiểm lâm) hoặc thậm chí là các loài chim, côn trùng bay sát ống kính. Nếu không có bộ lọc ngoài phân phối (Out-of-Distribution - OOD Detection), mô hình phân loại đóng (closed-set classifier) sẽ bị bắt buộc phải gán một trong 9 nhãn loài Sách Đỏ cho các đối tượng lạ này với độ tin cậy cao, gây ra các báo động giả nghiêm trọng. Do đó, việc thiết kế một cơ chế lọc thông minh dựa trên phân tích phân phối xác suất Softmax là bắt buộc để hệ thống hoạt động ổn định trong thực tế.

1.3 Mục tiêu nghiên cứu

Để mang lại một giải pháp khoa học thực tiễn hữu ích đóng góp cho công tác bảo tồn sinh thái và ứng dụng công nghệ số vào lâm nghiệp, đề tài hướng tới việc thực hiện trọn vẹn các mục tiêu cụ thể sau:

Mục tiêu thứ nhất là xây dựng thành công tập dữ liệu hình ảnh chuẩn hóa gồm 9 loài động vật hoang dã thuộc Sách Đỏ Việt Nam có tính đa dạng và cân bằng cao. Quy mô tập dữ liệu được thiết lập chính xác ở mức 300 hình ảnh chất lượng cao cho mỗi loài (tổng cộng 2.700 ảnh). Thông qua các kỹ thuật lọc sạch ảnh lỗi cơ bản và áp dụng pipeline tăng cường dữ liệu ở pha huấn luyện (bao gồm lật ảnh nằm ngang, xoay ảnh ngẫu nhiên từ -15 đến 15 độ, và hiệu chỉnh độ sáng tương phản ngẫu nhiên), nghiên cứu hướng tới loại bỏ hoàn toàn hiện tượng lệch lớp dữ liệu, tạo điều kiện lý thuyết tốt nhất để mô hình học máy học tập đặc trưng đồng đều mà không sinh ra lỗi dự đoán thiên lệch.

Mục tiêu này tập trung giải quyết câu hỏi nghiên cứu (Research Question 1 - RQ1): Làm thế nào để thu thập, làm sạch và chuẩn hóa dữ liệu hình ảnh của các loài động vật hoang dã quý hiếm tại Việt Nam vốn có số lượng ảnh cực kỳ hạn chế trên Internet và thực địa? Quy trình này đòi hỏi việc thiết kế một pipeline lọc nhiễu tự động, chuyển đổi kênh màu đồng nhất và áp dụng kỹ thuật sinh ảnh biến thể để cân bằng dữ liệu đầu vào mà không gây ra lỗi rò rỉ thông tin giữa các tập Train, Validation và Test.

Mục tiêu thứ hai là nghiên cứu, thiết kế và tối ưu hóa chiến lược học chuyển giao dựa trên mô hình nền ResNet-18 tiền huấn luyện trên ImageNet. Nhóm thực hiện đặt mục tiêu nghiên cứu và chứng minh tính hiệu quả của cơ chế đóng băng tham số (Parameter Freezing) đối với Layer 1, Layer 2 và cả đồng Layer 3, Layer 4 kết hợp lớp phân loại tùy biến có tích hợp BatchNorm và Dropout. Quá trình huấn luyện áp dụng hàm mất mát CrossEntropyLoss có Label Smoothing (0.1) cùng bộ tối ưu Adam với tốc độ học phân cấp (Differential Learning Rates) chạy qua 15 epoch để tối ưu hóa khả năng hội tụ nhanh của mạng, đạt độ chính xác validation vượt mốc 93% và độ chính xác kiểm thử độc lập vượt 95%.

Mục tiêu này tương ứng với câu hỏi nghiên cứu RQ2: Chiến lược huấn luyện nào giúp mô hình học sâu ResNet-18 đạt độ chính xác tối ưu trên tập dữ liệu nhỏ mà không bị hiện tượng quá khớp hay overfitting? Nghiên cứu sẽ tập trung khảo sát tác động của việc đóng băng một phần trọng số của mô hình ResNet-18 đã được pre-train trên ImageNet, kết hợp với việc áp dụng tốc độ học phân cấp (differential learning rates) và kỹ thuật làm mịn nhãn để tăng khả năng tổng quát hóa của bộ phân loại.

Mục tiêu thứ ba là ứng dụng thành công thuật toán Grad-CAM (Gradient-weighted Class Activation Mapping) nhằm giải thích trực quan hóa các quyết định phân loại của mô hình AI. Bằng cách tính đạo hàm ngược và nhân chập tuyến tính các bản đồ kích hoạt của Layer 4, hệ thống hướng tới xuất bản đồ nhiệt chồng chập lên ảnh gốc chỉ ra chính xác vùng đặc trưng vật thể mà AI tập trung nhiều nhất. Mục tiêu này giúp minh bạch hóa cơ chế ra quyết định của mạng nơ-ron sâu, nâng cao tính tin cậy khoa học của sản phẩm khi ứng dụng thực tế.

Mục tiêu này kiểm chứng giả thuyết nghiên cứu RQ3: Vùng chú ý trích xuất từ các lớp tích chập cuối cùng của ResNet-18 có thực sự tương đồng với các đặc trưng

sinh học nhận dạng bên ngoài của động vật hay không? Bằng cách triển khai giải thuật Grad-CAM, nhóm nghiên cứu sẽ phân tích định tính và định lượng xem các vùng pixel được kích hoạt mạnh nhất có nằm đúng vào thân hình, đầu, hay hoa văn của con vật hay không, qua đó đánh giá độ tin cậy thực sự của mô hình AI.

Mục tiêu thứ tư là thiết kế, xây dựng và đóng gói hoàn thiện hai phiên bản giao diện phần mềm đáp ứng nhu cầu thực tế: Ứng dụng Desktop chuyên nghiệp bằng CustomTkinter tích hợp bản đồ nhiệt Grad-CAM, biểu diễn Top-3 xác suất và hồ sơ động vật Sách Đỏ Việt Nam kèm bộ lọc ngưỡng tin cậy động 85%; và ứng dụng Web trực tuyến bằng Streamlit chạy suy luận luồng ảnh/video trên trình duyệt, tạo điều kiện thuận lợi nhất cho người dùng cuối tương tác.

Mục tiêu này giải quyết câu hỏi RQ4: Làm thế nào để đóng gói mô hình AI thành các ứng dụng phần mềm mượt mà, ổn định và dễ sử dụng cho cán bộ kiểm lâm không có chuyên môn về công nghệ? Giải pháp cần đảm bảo khả năng xử lý video thời gian thực không bị treo giao diện chính nhờ lập trình đa luồng (multithreading), đồng thời tích hợp các cơ chế tóm tắt phát hiện (detection grid summary) và bộ lọc động vật lạ (OOD filter).

1.4 Phạm vi và giới hạn đề tài

Để đảm bảo tính khả thi cao trong quá trình thiết kế, thực nghiệm và triển khai đề tài nghiên cứu khoa học trong phạm vi một học phần công nghệ chuyên sâu như Deep Learning, nhóm nghiên cứu đã thảo luận và xác lập rõ ràng giới hạn ranh giới chuyên môn cùng phạm vi kỹ thuật của hệ thống như sau:

Về phạm vi đối tượng nghiên cứu, hệ thống tập trung phân loại và nhận diện tối ưu đối với 9 loài động vật hoang dã nguy cấp tiêu biểu có tên trong Sách Đỏ Việt Nam và Danh lục đỏ IUCN, bao gồm: Báo Gấm, Cu Li Nhỏ, Gấu Ngựa, Hổ Đông Dương, Hươu Sao, Tê Tê Java, Voi Châu Á, Voọc Chà Vá, và Vượn Má Vàng. Đây là những loài động vật có giá trị bảo tồn sinh học và sinh thái rừng nhiệt đới vô cùng lớn nhưng số lượng cá thể ngoài tự nhiên đang bị đe dọa nghiêm trọng. Tập dữ liệu hình ảnh được thu thập, xử lý chuẩn hóa kích thước, và tăng cường dữ liệu ở pha huấn luyện

được thiết kế chuyên biệt để phục vụ việc trích xuất đặc trưng của 9 lớp đối tượng động vật này, không mở rộng ra các động vật phổ biến khác.

9 loài động vật này đại diện cho các mức độ đe dọa khác nhau trong Danh lục Đỏ IUCN và Sách Đỏ Việt Nam: Báo Gấm (EN - Nguy cấp), Cu Li Nhỏ (VU - Sắp nguy cấp), Gấu Ngựa (VU), Hồ Đông Dương (EN), Hươu Sao (EW - Tuyệt chủng ngoài tự nhiên/CR ở Việt Nam), Tê Tê Java (CR - Cực kỳ nguy cấp), Voi Châu Á (EN), Voọc Chà Vá Chân Nâu (CR), và Vượn Má Vàng (EN). Việc chọn các loài tiêu biểu này giúp bao phủ toàn bộ các nhóm đặc trưng hình thái khác nhau từ linh trưởng, thú ăn thịt cho đến loài có vảy sừng và động vật cỡ lớn.

Về giới hạn kỹ thuật và môi trường triển khai, hệ thống hiện tại được thiết kế tối ưu để nhận dạng và phân tích trên các tệp tin hình ảnh tĩnh (JPG, PNG, JPEG) hoặc tệp tin video tải lên trực tiếp từ thiết bị lưu trữ của người dùng cuối. Hệ thống chưa hỗ trợ chức năng kết nối trực tiếp với luồng truyền phát video RTSP thời gian thực từ các hệ thống camera giám sát thực địa hay máy bay ảnh tầm xa hoạt động trên quy mô mạng diện rộng của các khu bảo tồn quốc gia, do giới hạn hạ tầng truyền thông thực tế. Thêm vào đó, mô hình học sâu ResNet-18 sau huấn luyện được định cấu hình chạy suy luận trực tiếp trên nền tảng phần cứng PC tiêu chuẩn (CPU/GPU) thông qua các giao diện ứng dụng Desktop và Web. Nhóm nghiên cứu chưa tiến hành nén mô hình, lượng hóa trọng số hay tối ưu hóa mã nguồn để tích hợp chạy trực tiếp trên các chip vi xử lý biên có bộ nhớ và công suất cực kỳ giới hạn như máy ảnh thông minh dùng Raspberry Pi hay Jetson Nano ngoài thực địa rừng sâu.

Do đặc thù ảnh máy ảnh camera thường có chất lượng kém (bị rung mờ do con vật di chuyển nhanh, bị che khuất bởi tán lá, hoặc ảnh đen trắng ban đêm sử dụng đèn hồng ngoại), mô hình sẽ gặp khó khăn lớn trong việc nhận diện nếu con vật chỉ xuất hiện một phần nhỏ cơ thể ở góc khuất. Đề tài giới hạn phạm vi nghiên cứu ở việc nhận diện các ảnh/video chứa các góc chụp tương đối rõ nét của động vật và chưa tích hợp các thuật toán theo dõi đối tượng liên tục nâng cao trong môi trường rừng rậm rạp.

1.5 Công nghệ và ngôn ngữ lập trình chính

Để đảm bảo tính ổn định, hiệu năng xử lý cao, tốc độ suy luận nhanh và khả năng mở rộng tốt trong tương lai, nhóm nghiên cứu đã lựa chọn và kết hợp đồng bộ hệ sinh thái các công nghệ mã nguồn mở phổ biến trong cộng đồng nghiên cứu Trí tuệ Nhân tạo hiện nay. Các công nghệ cốt lõi bao gồm:

Đầu tiên là ngôn ngữ lập trình chủ đạo Python phiên bản 3.10+. Python được lựa chọn nhờ sở hữu cú pháp ngắn gọn, dễ tiếp cận và được hỗ trợ bởi một cộng đồng khoa học dữ liệu khổng lồ. Ngôn ngữ này đóng vai trò chất keo kết dính, liên kết toàn bộ pipeline của dự án từ khâu thu thập ảnh thô, tiền xử lý ma trận điểm ảnh, huấn luyện mô hình sâu bằng PyTorch cho đến việc lập trình xây dựng các giao diện ứng dụng.

Để quản lý thư viện hiệu quả và tránh xung đột phiên bản, dự án sử dụng công cụ tạo môi trường ảo venv riêng biệt cho toàn bộ hệ thống. Mọi gói phụ thuộc được khai báo rõ ràng trong tệp requirements.txt giúp đảm bảo khả năng tái lập của mã nguồn huấn luyện và triển khai ứng dụng trên các máy tính khác nhau một cách dễ dàng.

Thứ hai là framework học sâu chính PyTorch cùng thư viện hỗ trợ TorchVision. PyTorch được ưu tiên sử dụng thay vì các framework khác do cơ chế đồ thị tính toán động (Dynamic Computation Graph). Cơ chế này cho phép nhóm nghiên cứu dễ dàng can thiệp trực tiếp vào mô hình bằng cách đăng ký các hàm Hook (đạo hàm ngược), từ đó bắt được các kích hoạt và đạo hàm riêng của lớp tích chập cuối cùng phục vụ đặc lực cho việc lập trình thuật toán giải thích mô hình Grad-CAM. TorchVision cung cấp sẵn kiến trúc mạng ResNet-18 đã được tiền huấn luyện trên ImageNet cùng các module biến đổi Tensor ảnh vô cùng tối ưu.

Mặc dù TensorFlow là một framework mạnh mẽ và phổ biến trong công nghiệp, PyTorch lại được ưu tiên lựa chọn nhờ triết lý thiết kế hướng lập trình viên (pythonic). Cơ chế đồ thị tính toán động (Dynamic Computation Graph) của PyTorch cho phép thay đổi kiến trúc mạng linh hoạt khi huấn luyện và gỡ lỗi dễ dàng hơn. Hơn nữa, việc đăng ký các hàm hook (register_forward_hook, register_backward_hook) để trích xuất

gradient phục vụ Grad-CAM được PyTorch hỗ trợ trực quan và hiệu quả hơn rất nhiều so với cơ chế lưu trữ đồ thị tính của TensorFlow.

Thứ ba là các thư viện xử lý ảnh chuyên sâu bao gồm OpenCV (Open Source Computer Vision Library) và Pillow. OpenCV được sử dụng để xử lý luồng khung hình video từ các camera giám sát tĩnh và áp dụng các phép tính toán ma trận để chồng chấp bản đồ nhiệt Grad-CAM lên ảnh gốc với độ trong suốt 40%. Pillow đóng vai trò trung gian trong việc định dạng, resize và cắt cúp hình ảnh để giữ nguyên tỷ lệ động vật ban đầu, tránh làm biến dạng cơ thể động vật dẫn đến sụt giảm độ tự tin nhận diện.

Sự kết hợp giữa OpenCV và Pillow mang lại hiệu quả tối ưu: OpenCV được dùng cho các tác vụ xử lý ma trận ảnh ở mức độ thấp (low-level matrix operations) có hiệu năng cực cao như giải mã video thành khung hình, đổi không gian màu, vẽ khung bao và phối trộn ảnh nhiệt; trong khi Pillow lại đảm nhận các tác vụ xử lý ảnh ở mức độ cao (high-level image operations) như mở tệp định dạng phức tạp, chuyển đổi chế độ màu (RGBA sang RGB) và chuẩn bị định dạng dữ liệu đầu vào phù hợp cho lớp tiền xử lý của TorchVision.

Thứ tư là các thư viện xây dựng giao diện ứng dụng chuyên nghiệp. Giao diện Desktop được phát triển bằng CustomTkinter – một thư viện kế thừa Tkinter của Python nhưng bổ sung các widget phẳng hiện đại, hỗ trợ giao diện tối thời thượng. Giao diện Web được phát triển bằng Streamlit – một framework đám mây tối giản giúp nhanh chóng đóng gói mã nguồn suy luận của AI thành trang web trực quan chạy trên các trình duyệt mà không cần lập trình HTML/CSS/JS phức tạp.

CustomTkinter mang lại giao diện GUI hiện đại (hỗ trợ Dark Mode, thiết kế phẳng thời thượng) nhưng vẫn giữ được sự gọn nhẹ và ổn định nguyên bản của Tkinter trên hệ điều hành Windows, rất phù hợp cho việc đóng gói thành phần mềm Desktop chạy offline cho kiểm lâm viên. Ngược lại, Streamlit được chọn cho ứng dụng Web nhờ khả năng chuyển đổi mã nguồn Python thành ứng dụng Web trực tuyến chỉ trong vài chục dòng lệnh mà không cần kiến thức về HTML/CSS/Javascript, giúp nhanh chóng chia sẻ sản phẩm thử nghiệm cho các nhà khoa học và cộng đồng bảo tồn trực tuyến.

CHƯƠNG 2: KIẾN TRÚC HỆ THỐNG VÀ CÁC PHÂN HỆ CHÍNH

2.1 Sơ đồ luồng hoạt động hệ thống

Hệ thống nhận diện động vật Sách Đỏ được nhóm nghiên cứu thiết kế theo dạng đường ống khép kín (End-to-End Pipeline) tuần tự và tối ưu hóa ở từng khâu. Kiến trúc này giúp đảm bảo luồng đi của dữ liệu được quản lý chặt chẽ và không xảy ra các hiện tượng rò rỉ dữ liệu hoặc sai lệch phân phối làm ảo hóa kết quả học máy. Toàn bộ pipeline được tổ chức làm 5 giai đoạn liên đới bao gồm:



Hình 2.1: Sơ đồ luồng hoạt động End-to-End của hệ thống nhận diện động vật Sách Đỏ.

Giai đoạn 1: Kỹ nghệ Dữ liệu. Đây là bước đầu tiên để xây dựng tập dữ liệu thô. Dữ liệu hình ảnh được thu thập tự động từ internet bằng script `'crawl_bing_images.py'`. Nhóm tiến hành lọc sạch các tệp lỗi, hỏng cấu trúc bằng `'clean_dataset.py'` và khôi phục dữ liệu gốc. Ngay sau đó, script `'canbang.py'` được gọi để tự động phát hiện số lượng ảnh bị thiếu của từng loài và áp dụng các thuật toán tăng cường ảnh (Image Augmentation) để nhân bản dữ liệu, đưa số lượng ảnh của mỗi loài đạt chuẩn đồng đều 300 ảnh/loài trên đĩa cứng nhằm giải quyết bài toán mất cân bằng lớp (Class Imbalance).

Trong giai đoạn này, dữ liệu trải qua sự biến đổi về mặt cấu trúc lưu trữ và kiểu dữ liệu. Ban đầu, hình ảnh được tải về dưới dạng các tệp tin nhị phân tĩnh (JPG, PNG) lưu trữ cục bộ trên ổ đĩa. Khi script crawl hoạt động, nó sẽ kiểm tra định dạng tệp bằng cách đọc cấu trúc byte tiêu đề (header bytes) để loại bỏ các tệp tin hỏng hoặc tải thiếu. Hình ảnh hợp lệ được tổ chức vào các thư mục con tương ứng với tên loài động vật trong hệ thống tệp tin NTFS/ext4, chuẩn bị cho bước làm sạch tiếp theo.

Giai đoạn 2: Tiền xử lý & Phân tích Dữ liệu (Data Preprocessing & Analysis). Tập dữ liệu gốc sau khi dọn dẹp sạch sẽ được chia thành 3 phần độc lập: tập huấn luyện (Train) chiếm 70%, tập kiểm thử tinh chỉnh (Validation) chiếm 20% và tập đánh giá độc lập (Test) chiếm 10% thông qua kịch bản `'tienxulyanh.py'`. Việc chia dữ liệu này được thực hiện trên tập dữ liệu gốc ban đầu chưa thực hiện augmentation để đảm bảo tính độc lập tuyệt đối, tránh hiện tượng rò rỉ dữ liệu (Data Leakage) khi huấn luyện. Sau khi chia tập, các phép biến đổi chuẩn hóa (Resize về 224x224, Normalize theo phân phối chuẩn của ImageNet) được thiết lập để đưa ảnh về dạng tensor thích hợp với mạng ResNet-18.

Tại đây, dữ liệu chuyển đổi từ dạng tệp tin hình ảnh thô sang các cấu trúc mảng trong bộ nhớ. Sau khi dọn dẹp và phân chia, hình ảnh được nạp bởi thư viện Pillow dưới dạng ảnh 3 kênh màu RGB. Khi đi qua pipeline tiền xử lý của torchvision, ảnh RGB được thay đổi kích thước về 256 x 256 pixel bằng thuật toán nội suy song tuyến tính, cắt trung tâm thành 224 x 224 pixel và chuyển đổi thành Tensor PyTorch có kích thước (shape) [3, 224, 224] với các giá trị kiểu số thực float32 được chuẩn hóa về

khoảng [0.0, 1.0]. Cuối cùng, các giá trị này được trừ đi giá trị trung bình (mean) và chia cho độ lệch chuẩn (std) của tập ImageNet để chuẩn hóa phân phối dữ liệu đầu vào.

Giai đoạn 3: Huấn luyện Mô hình Học sâu (Model Training). Phân hệ này sử dụng mô hình nền tảng ResNet-18 đã được tiền huấn luyện trên cơ sở dữ liệu ImageNet làm điểm khởi đầu (Transfer Learning) nhằm tận dụng khả năng trích xuất đặc trưng hình học sẵn có. Nhóm áp dụng kỹ thuật tinh chỉnh (Fine-tuning) bằng cách đóng băng tham số của Layer 1 và Layer 2, rã đông Layer 3 và Layer 4 nhằm học các đặc trưng cấp cao đặc thù của động vật hoang dã. Lớp phân loại cuối cùng (Fully Connected) được thay thế hoàn toàn bằng một Classifier tùy biến có tích hợp Batch Normalization, ReLU và Dropout ($p=0.5$). Quá trình huấn luyện sử dụng hàm sai số CrossEntropyLoss tích hợp Label Smoothing (0.1) cùng bộ tối ưu Adam với cơ chế tốc độ học phân cấp (Differential Learning Rates) chạy qua 15 epoch.

Trong pha huấn luyện, tensor hình ảnh [3, 224, 224] được gom thành các batch có kích thước [batch_size, 3, 224, 224] (thường chọn batch_size = 32 hoặc 64) để đưa vào mạng ResNet-18. Mạng nơ-ron thực hiện lan truyền xuôi qua các tầng tích chập để trích xuất đặc trưng hình thái, biến đổi tensor đầu vào thành một vector đặc trưng kích thước 512 chiều tại đầu ra của Global Average Pooling layer. Vector này tiếp tục đi qua Custom Fully Connected Layer (Classifier) để tính ra vector logit có kích thước [batch_size, 9] đại diện cho điểm số chưa chuẩn hóa của 9 lớp loài động vật mục tiêu. Hàm mất mát CrossEntropyLoss sau đó sẽ so sánh vector logit này với nhãn đúng để tính toán giá trị tổn thất (loss value) phục vụ lan truyền ngược cập nhật gradient.

Giai đoạn 4: Đánh giá Chất lượng Mô hình (Model Evaluation). Kịch bản `'phantichthongso.py'` tải bộ trọng số tối ưu nhất thu được từ pha huấn luyện (`'ResNet18BestWeights.pth'`) để chạy suy luận trên tập Test độc lập. Hệ thống tính toán các chỉ số toán học chuẩn như Accuracy toàn cục, Recall trung bình, và vẽ các đồ thị ROC-AUC đa lớp cho cả 9 loài động vật sặc đỏ. Việc kiểm tra này giúp đánh giá năng lực suy luận thực tế của AI đối với các ảnh nằm ngoài tầm kiểm soát huấn luyện.

Pha đánh giá sử dụng dữ liệu kiểm thử (Test set) chưa từng xuất hiện trong quá trình huấn luyện. Mô hình chạy ở chế độ eval (`model.eval()`) và tắt tính toán gradient (`torch.no_grad()`) để tiết kiệm bộ nhớ và tăng tốc độ suy luận. Vector logit [batch_size,

9] đầu ra của mô hình được đi qua hàm Softmax để chuyển đổi thành phân phối xác suất [batch_size, 9], trong đó tổng xác suất của 9 lớp bằng 1. Lớp có xác suất cao nhất được chọn làm nhãn dự đoán. Các kết quả dự đoán này được tích lũy lại để tính toán ma trận nhầm lẫn (Confusion Matrix) kích thước 9 x 9 và các chỉ số Precision, Recall, F1-score bằng thư viện scikit-learn.

Giai đoạn 5: Triển khai Ứng dụng Sản phẩm (App Deployment). Bước cuối cùng đưa tri thức của mô hình vào thực tiễn cuộc sống. Bộ trọng số tối ưu được tích hợp vào ứng dụng Desktop chuyên nghiệp viết bằng CustomTkinter (`app\_desktop.py`) có giao diện Dark Mode hiện đại. Giao diện này hiển thị xác suất top 3 loài dự đoán, hồ sơ thông tin sách đỏ của loài đó và chồng bản đồ nhiệt Grad-CAM lên ảnh gốc để người dùng thấy vùng chú ý của AI. Ngoài ra, giao diện Web Streamlit (`appnhandien.py`) và kịch bản video nhanh (`run\_video\_inference.py`) cũng được triển khai để tăng tính đa dạng trong môi trường sử dụng thực tế.

Trong pha triển khai, bộ trọng số tốt nhất (45MB) được nạp vào mô hình ResNet-18 trên thiết bị đầu cuối của người dùng. Khi người dùng tải lên một hình ảnh hoặc luồng video thông qua ứng dụng Desktop (CustomTkinter) hoặc Web (Streamlit), khung hình sẽ được tiền xử lý giống hệt như pha huấn luyện để tạo thành tensor [1, 3, 224, 224]. Mô hình thực hiện suy luận nhanh (inference time khoảng 15-30ms) để đưa ra phân phối xác suất của 9 loài động vật hoang dã. Nếu lớp có xác suất cao nhất vượt qua ngưỡng OOD (85%), ứng dụng sẽ hiển thị nhãn loài tương ứng kèm thông tin sinh học và bản đồ nhiệt Grad-CAM giúp người dùng dễ dàng kiểm chứng.

Bảng 2.1: Bảng phân công nhiệm vụ và cấu trúc đóng góp của các thành viên.

Thành viên thực hiện	Vai trò chính	Mã nguồn phụ trách	Nội dung đóng góp chủ yếu
Đặng Thành Thi (Nhóm trưởng)	Kỹ sư Dữ liệu & Phát triển Ứng dụng	crawl_bing_images.py canbang.py app_desktop.py appnhandien.py	Xây dựng mã nguồn thu thập dữ liệu tự động, thực hiện tăng cường

			dữ liệu (augmentation) và cân bằng lớp; phát triển phần mềm Desktop bằng CustomTkinter và ứng dụng Web bằng Streamlit.
Lưu Đức Linh (Thành viên)	Nhà nghiên cứu Mô hình AI & Huấn luyện	resnet18finetuning.py train_resnet18_full.py	Thiết kế kiến trúc mô hình ResNet-18 (Transfer Learning), cấu hình đóng băng/giải băng các lớp mạng và tối ưu hóa quá trình huấn luyện.
Trần Xuân Hương (Thành viên)	Kỹ sư Phân tích Dữ liệu & Đánh giá Hiệu năng	clean_dataset.py tienxulyanh.py phantichdulieu.py phantichthongso.py	Thực hiện tiền xử lý và lọc nhiễu dữ liệu, phân chia tập Train/Val/Test độc lập tránh rò rỉ thông tin; đánh giá hiệu năng mô hình qua biểu đồ Loss/Accuracy và đường cong ROC-AUC.

2.2 Phân hệ Kỹ thuật Dữ liệu

Phân hệ Kỹ thuật Dữ liệu đóng vai trò nền móng thiết lập chất lượng cho toàn bộ hệ thống AI. Trong các bài toán học sâu, đặc biệt là thị giác máy tính thực tế, chất lượng dữ liệu quyết định trực tiếp giới hạn trên của hiệu năng mô hình. Do tính chất đặc thù của các loài động vật hoang dã nguy cấp có tên trong Sách Đỏ, việc thu thập hình ảnh tự nhiên đối mặt với sự khan hiếm nghiêm trọng cùng sự phân bố không đồng đều giữa các loài. Để giải quyết, phân hệ được thiết kế gồm ba module chính phối hợp khép kín trên đĩa cứng:

Module thu thập dữ liệu tự động: Script này sử dụng các thư viện BeautifulSoup4 và Requests để tự động gửi yêu cầu cào dữ liệu hình ảnh từ công cụ tìm kiếm Bing. Quá trình cào ảnh được định hướng bởi các bộ từ khóa song ngữ Anh - Việt mô tả chính xác tên khoa học và tên thường gọi của từng loài (ví dụ: 'Clouded leopard Neofelis nebulosa', 'Vọc chà vá', 'Java pangolin Manis javanica'). Sau khi lọc các đường liên kết ảnh hợp lệ, kịch bản tự động tải các tệp ảnh về các thư mục tương ứng của 9 loài, tạo ra cơ sở dữ liệu thô ban đầu có phong nền, góc chụp, và kích thước vô cùng đa dạng.

Về chi tiết lập trình, script `crawl_bing_images.py` thực hiện mô phỏng hành vi của trình duyệt người dùng bằng cách chèn tiêu đề User-Agent hợp lệ (như Mozilla/5.0) vào các yêu cầu HTTP GET. Điều này giúp ngăn chặn cơ chế phát hiện bot và chặn IP tự động từ máy chủ tìm kiếm Bing. Script sử dụng BeautifulSoup4 để phân tích cú pháp HTML trả về, trích xuất thuộc tính hình ảnh nguồn từ các thẻ `img` thích hợp. Quá trình tải xuống ảnh được thực hiện trong một vòng lặp có cơ chế xử lý ngoại lệ (`try-except`) chặt chẽ: nếu một liên kết ảnh bị lỗi kết nối hoặc phản hồi quá lâu (`timeout > 10` giây), script sẽ tự động bỏ qua và chuyển sang liên kết tiếp theo để đảm bảo luồng chạy không bị gián đoạn. Mỗi ảnh tải về đều được thư viện Pillow kiểm tra tính hợp lệ bằng cách cố gắng mở tệp (`Image.open()`); nếu tệp bị lỗi định dạng hoặc không thể đọc, nó sẽ bị xóa ngay lập tức.

Module dọn dẹp và reset dữ liệu: Nhằm đảm bảo dữ liệu không chứa các tệp lỗi có thể làm sụp đổ tiến trình huấn luyện nơ-ron kéo dài nhiều giờ trên GPU, kịch bản này quét qua toàn bộ thư mục dữ liệu thô. Nó sử dụng thư viện Pillow mở kiểm tra cấu trúc

từng tệp ảnh. Các tệp ảnh bị hỏng định dạng vật lý (corrupted files), tệp có dung lượng bằng 0 byte, hoặc định dạng không được hỗ trợ sẽ bị tự động xóa bỏ hoàn toàn khỏi ổ đĩa. Hơn thế nữa, đối với các ảnh bị lỗi cấu trúc nhẹ nhưng vẫn có thể đọc được, nhóm nghiên cứu đã thiết lập cờ `ImageFile.LOAD_TRUNCATED_IMAGES = True` để cho phép PyTorch tiếp tục tải ảnh mà không ném ra ngoại lệ runtime `OSError: image file is truncated` làm gián đoạn chương trình.

Về cơ chế hoạt động, script `clean_dataset.py` sử dụng hàm `os.walk()` để duyệt đệ quy qua toàn bộ cây thư mục dữ liệu. Với mỗi tệp tin tìm thấy, script kiểm tra kích thước tệp bằng hàm `os.path.getsize()`. Những tệp tin có kích thước quá nhỏ (dưới 5KB) thường là các ảnh thumbnail chất lượng cực kém hoặc tệp tin tải xuống bị lỗi giữa chừng, sẽ được liệt kê vào danh sách cần loại bỏ. Tiếp theo, script mở từng tệp tin bằng Pillow và thực hiện phép biến đổi thử nghiệm (như chuyển hệ màu); nếu phát sinh lỗi `OSError` hoặc `UnidentifiedImageError`, tệp tin đó được xác định là bị hỏng cấu trúc byte bên trong. Script triển khai chế độ chạy thử nghiệm (dry-run mode) hiển thị danh sách các tệp tin lỗi và tổng dung lượng giải phóng trước khi người dùng xác nhận thực hiện xóa vĩnh viễn bằng `os.remove()`, giúp tránh việc vô tình xóa nhầm dữ liệu quan trọng.

Module cân bằng dữ liệu (`canbang.py`): Giải quyết triệt để bài toán lệch lớp (Class Imbalance) - một trong những nguyên nhân hàng đầu khiến mô hình AI bị học vẹt hoặc thiên lệch dự đoán về các lớp chiếm đa số. Đối với các loài cực kỳ nguy cấp có lượng ảnh cao được rất ít (dưới 300 ảnh), module này tự động đếm số lượng ảnh hiện có và thực hiện bù đắp. Bằng việc áp dụng một chuỗi các phép biến đổi hình học và màu sắc ngẫu nhiên (Offline Image Augmentation) bao gồm lật ngang ngẫu nhiên, xoay ảnh từ -15 đến 15 độ, và điều chỉnh ngẫu nhiên độ tương phản, độ sáng bằng `ColorJitter`, kịch bản sinh ra các phiên bản ảnh biến thể tự nhiên cho đến khi mỗi loài có đủ chính xác 300 hình ảnh.

Về mặt kỹ thuật lập trình, module `canbang.py` được thiết kế theo mô hình kiểm tra điều kiện trước khi hành động (pre-condition check pattern). Trước khi thực hiện bất kỳ phép biến đổi tăng cường nào, script sẽ đếm chính xác số lượng tệp ảnh hiện có trong từng thư mục loài bằng hàm `os.listdir()` kết hợp bộ lọc đuôi tệp (`.jpg`, `.png`, `.jpeg`).

Chỉ khi số lượng ảnh hiện tại thấp hơn ngưỡng mục tiêu (210 ảnh cho tập Train), module mới kích hoạt vòng lặp sinh ảnh biến thể. Mỗi ảnh biến thể được đặt tên theo quy ước chuẩn: `AUG_{tên_ảnh_gốc}_{loại_biến_đổi}.jpg`, giúp phân biệt rõ ràng giữa dữ liệu gốc và dữ liệu tăng cường trong quá trình gỡ lỗi và kiểm tra chất lượng sau này.

Kịch bản `canbang.py` hoạt động bằng cách đếm số lượng tệp ảnh hiện có trong thư mục Train của từng loài và so sánh với con số mục tiêu 210 ảnh. Nếu số lượng ảnh thực tế của một loài nhỏ hơn 210, script sẽ tính toán số lượng ảnh cần sinh thêm ($N_{need} = 210 - N_{current}$). Vòng lặp tăng cường dữ liệu sẽ chọn ngẫu nhiên các ảnh gốc của loài đó, áp dụng các phép biến đổi hình học và màu sắc ngẫu nhiên của thư viện Pillow (như `Image.transpose(Image.FLIP_LEFT_RIGHT)` để lật ngang, `Image.rotate` để xoay góc từ -15 đến +15 độ, hoặc sử dụng `ImageEnhance` để thay đổi độ sáng và độ tương phản từ 0.8 đến 1.2 lần) để tạo ra ảnh mới và lưu lại với tiền tố `AUG_`. Quy trình này dừng lại ngay khi số lượng ảnh đạt đúng 210, đảm bảo tính cân bằng hoàn hảo giữa các lớp trong tập huấn luyện.

Một điểm kỹ thuật quan trọng cần nhấn mạnh là module `canbang.py` chỉ được gọi sau khi quá trình phân chia tập Train/Val/Test đã hoàn tất. Điều này có nghĩa là các ảnh augment chỉ tồn tại trong thư mục con Train, hoàn toàn cách ly với các thư mục Validation và Test. Cơ chế cách ly vật lý trên hệ thống tệp (filesystem-level isolation) này là rào cản mạnh nhất chống lại mọi hình thức rò rỉ dữ liệu, vượt trội hơn so với các phương pháp cách ly chỉ dựa trên chỉ mục trong bộ nhớ RAM thường gặp trong các dự án học máy sinh viên.

Nhóm nghiên cứu đã thảo luận kỹ lưỡng để chọn ra các thông số tăng cường ảnh hợp lý: việc lật ngang giúp nhân đôi góc nhìn của động vật; việc xoay nhẹ trong khoảng -15 đến 15 độ mô phỏng tư thế nghiêng đầu tự nhiên của con thú mà không làm đảo lộn cấu trúc giải phẫu sinh học của chúng; và bộ lọc `ColorJitter` với biên độ 15% giúp mô hình thích nghi tốt với sự biến thiên cường độ ánh sáng của rừng rậm nhiệt đới. Việc cân bằng lớp vật lý này tạo điều kiện lý thuyết tốt nhất để mô hình no-ron học tập đồng đều các đặc trưng của cả 9 loài động vật hoang dã.

2.3 Phân hệ Huấn luyện & Đánh giá mô hình AI (Model Pipeline)

Phân hệ Huấn luyện & Đánh giá mô hình AI là trung tâm tính toán chính, nơi chuyển hóa dữ liệu hình ảnh thành tri thức trích xuất đặc trưng của mạng nơ-ron tích chập sâu. Phân hệ được thiết kế khoa học thông qua sự phối hợp chặt chẽ của ba script mã nguồn chính:

Thiết lập cấu trúc tinh chỉnh mạng: Khối mã nguồn này chịu trách nhiệm nạp cấu trúc mạng ResNet-18 đã được tiền huấn luyện trên ImageNet. Nhóm áp dụng chiến lược tinh chỉnh tham số phân cấp bằng cách khóa trọng số của Layer 1 và Layer 2 để giữ nguyên các bộ trích xuất đặc trưng cơ bản (biên cảnh, cấu trúc hình học, màu sắc). Đồng thời, rã đông Layer 3 và Layer 4 nhằm cho phép chúng học các đặc trưng cấp cao đặc thù của động vật hoang dã. Bộ phân loại Fully Connected cũ được thay thế bằng một Custom Classifier gồm: lớp Linear(512, 256), lớp BatchNorm1d để chuẩn hóa dữ liệu tăng tốc độ hội tụ, hàm kích hoạt phi tuyến tính ReLU và lớp Dropout với xác suất $p=0.5$ để tắt ngẫu nhiên 50% nơ-ron giúp mô hình chống hiện tượng học vẹt overfitting.

Chi tiết hơn về mặt kiến trúc mạng, ResNet-18 bao gồm tổng cộng khoảng 11.17 triệu tham số. Với chiến lược đóng băng Layer 1 và Layer 2, số lượng tham số thực sự tham gia vào quá trình cập nhật gradient chỉ còn khoảng 5.6 triệu tham số (bao gồm Layer 3, Layer 4 và Custom Classifier). Việc giảm 50% số tham số cần huấn luyện mang lại hai lợi ích song song: thứ nhất, giảm đáng kể dung lượng bộ nhớ GPU cần thiết (từ khoảng 1.2GB xuống còn 0.7GB cho mỗi batch size 32), cho phép chạy huấn luyện trên các GPU phổ thông GTX 1650 4GB; thứ hai, giảm thiểu nguy cơ quá khớp trên tập dữ liệu nhỏ do không gian giả thuyết (hypothesis space) bị thu hẹp, buộc mô hình phải học các đặc trưng thực sự mang tính phân biệt thay vì ghi nhớ nhiễu.

Về cấu trúc lập trình trong `resnet18finetuning.py`, mô hình ResNet-18 pre-trained được tải về bằng hàm `torchvision.models.resnet18(weights=docx.shared.Pt if hasattr(docx.shared, 'Pt') else None)`. Wait, let me use standard python syntax: `torchvision.models.resnet18(weights=ResNet18_Weights.DEFAULT)`. Sau đó, script duyệt qua toàn bộ các tham số của mô hình và đặt thuộc tính `requires_grad = False` để đóng băng trọng số. Riêng đối với Layer 3 và Layer 4 (`layer3` và `layer4`), script mở băng bằng cách đặt lại `requires_grad = True` cho tất cả các lớp tích chập nằm trong các

khối residual blocks này. Lớp phân loại cuối cùng (model.fc) ban đầu có số lượng đầu ra là 1000 lớp (theo chuẩn ImageNet) được thay thế hoàn toàn bằng một lớp tuyến tính mới (nn.Linear(512, 9)) được khởi tạo ngẫu nhiên, chịu trách nhiệm học cách ánh xạ các đặc trưng đặc trưng 512 chiều thành 9 lớp động vật Sách Đỏ.

Về cơ chế lưu trữ trọng số tốt nhất, script `train_resnet18_full.py` triển khai một hệ thống giám sát hiệu năng validation theo thời gian thực. Sau mỗi epoch, hệ thống so sánh độ chính xác validation hiện tại với giá trị tốt nhất đã ghi nhận trước đó (`best_val_accuracy`). Nếu giá trị mới vượt qua kỷ lục cũ, toàn bộ trạng thái trọng số của mô hình sẽ được tuần tự hóa và ghi đè lên tệp tin `ResNet18_Best_Weights.pth` bằng hàm `torch.save()`. Cơ chế Early Stopping ngầm định này đảm bảo rằng bộ trọng số cuối cùng được sử dụng cho pha đánh giá Test không phải là trọng số ở epoch cuối cùng (có thể đã bắt đầu overfitting), mà là trọng số ở thời điểm mô hình đạt hiệu năng tổng quát hóa cao nhất trên tập dữ liệu validation hoàn toàn độc lập.

Quy trình huấn luyện và tối ưu hóa (`train_resnet18_full.py`): Script này chạy vòng lặp huấn luyện chính gồm 15 Epoch, mỗi epoch chia làm hai pha train và val tuần tự. Ở pha train, dữ liệu được nạp theo từng lô (Batch Size = 32), thực hiện lan truyền xuôi để tính toán hàm mất mát CrossEntropyLoss tích hợp Label Smoothing (0.1). Quá trình lan truyền ngược tính đạo hàm và cập nhật trọng số thông qua bộ tối ưu hóa Adam có tốc độ học phân cấp: Layer 3 và 4 học rất chậm ($1e-5$) để không phá vỡ tri thức cũ, trong khi lớp Classifier mới học nhanh hơn ($1e-3$). Ở pha val, mô hình chạy suy luận trên dữ liệu validation độc lập; nếu độ chính xác validation đạt kỷ lục mới, hệ thống tự động lưu trữ trạng thái trọng số tốt nhất vào tệp tin `ResNet18BestWeights.pth`.

Trong mã nguồn `train_resnet18_full.py`, lớp `DataLoader` của PyTorch đóng vai trò quản lý việc tải dữ liệu đa luồng hiệu quả. Các tham số như `num_workers=4` và `pin_memory=True` được thiết lập để tối ưu hóa việc truyền dữ liệu từ RAM hệ thống vào VRAM của GPU, giảm thiểu thời gian chờ của nhân CUDA. Script triển khai bộ tối ưu hóa Adam (`torch.optim.Adam`) với tốc độ học phân cấp (differential learning rates) được định nghĩa thông qua cấu trúc danh sách tham số truyền vào optimizer: `[{'params': model.layer3.parameters(), 'lr': 1e-5}, {'params': model.layer4.parameters(),`

'lr': 1e-5}, {'params': model.fc.parameters(), 'lr': 1e-3}]. Việc giám sát tổn thất huấn luyện và độ chính xác validation được thực hiện sau mỗi epoch để phát hiện sớm hiện tượng quá khớp và tự động kích hoạt ghi nhận bộ trọng số tốt nhất.

Phân tích đánh giá và kiểm thử khoa học (phantichthongso.py): Script này được gọi sau khi quá trình huấn luyện hoàn tất. Nó nạp lại tệp trọng số tốt nhất và chạy suy luận trên tập Test độc lập (AI chưa từng thấy trong suốt quá trình học và validation). Script tính toán các chỉ số đánh giá chuyên sâu bao gồm Accuracy toàn cục, Recall, Precision cho từng loài và vẽ đồ thị ROC-AUC đa lớp cùng biểu đồ biến thiên Loss/Accuracy qua 15 epoch để chứng minh tính khoa học và độ tin cậy của mô hình.

Về mặt lập trình, phantichthongso.py sử dụng thư viện matplotlib để tự động tạo ra hai tệp đồ thị quan trọng: ROC_AUC_Curve.png biểu diễn đường cong đặc trưng hoạt động của bộ thu nhận (Receiver Operating Characteristic) cho từng lớp loài động vật kèm chỉ số diện tích dưới đường cong (AUC), và Loss_Accuracy_Curves.png biểu diễn sự thay đổi của tổn thất và độ chính xác trên tập Train và Validation qua 15 epoch. Kịch bản sử dụng các hàm metrics.confusion_matrix, metrics.classification_report của scikit-learn để tính toán ma trận nhầm lẫn và kết xuất các chỉ số đánh giá vĩ mô (macro-average metrics) dưới dạng văn bản và lưu lại vào báo cáo, giúp đảm bảo tính minh bạch và khách quan tối đa của kết quả nghiên cứu khoa học.

2.4 Phân hệ Phát triển Giao diện Ứng dụng

Phân hệ Phát triển Giao diện Ứng dụng là giai đoạn cuối cùng trong pipeline phát triển phần mềm AI, đảm nhận vai trò đưa tri thức mạng nơ-ron tích chập ra khỏi môi trường lập trình khô khan để phục vụ tương tác trực quan cho người dùng cuối. Phân hệ được thiết kế đa dạng với ba thành phần giao diện chuyên biệt:

Ứng dụng Desktop chuyên nghiệp: Được phát triển trên nền tảng thư viện CustomTkinter với phong cách thiết kế tối hiện đại. Ứng dụng này nạp tệp trọng số tối ưu nhất và đưa mô hình về chế độ suy luận suy giảm tính toán (model.eval()). Khi người dùng tải ảnh lên, ứng dụng chạy song song hai luồng: phân loại động vật hiển thị top 3 loài có xác suất cao nhất kèm hồ sơ thông tin sinh thái từ Sách Đỏ Việt Nam;

và chạy thuật toán Grad-CAM trích xuất bản đồ kích hoạt của Layer 4 để hiển thị quảng sáng heatmap màu đỏ ám chỉ rõ vùng chú ý của AI. Ngoài ra, giao diện tích hợp một thanh trượt điều chỉnh ngưỡng tin cậy (Confidence Threshold Slider, mặc định 85%) để tự động cảnh báo 'loài lạ' khi điểm số dự đoán của mô hình quá thấp, bảo vệ hệ thống khỏi các nhận diện sai lệch.

Về mặt thiết kế luồng xử lý trong `app_desktop.py`, ứng dụng sử dụng cấu trúc Grid Layout của CustomTkinter để phân tách màn hình thành các khu vực chức năng trực quan: thanh điều khiển bên trái (Sidebar) chứa nút tải tệp tin, chọn nguồn camera và thanh điều chỉnh ngưỡng tin cậy; khu vực trung tâm hiển thị khung hình ảnh/video trực tiếp; và bảng thông tin bên phải hiển thị kết quả phân loại top 3 loài có độ tự tin cao nhất cùng bản đồ nhiệt Grad-CAM. Để tránh tình trạng giao diện bị đóng băng (Not Responding) khi mô hình AI đang suy luận trên các khung hình video liên tục, script sử dụng thư viện threading của Python để chạy tiến trình đọc khung hình và suy luận mô hình trên một luồng phụ riêng biệt, truyền kết quả hiển thị về luồng giao diện chính thông qua một hàng đợi thread-safe.

Ứng dụng Web trực tuyến Streamlit: Cung cấp giải pháp đám mây gọn nhẹ, cho phép người dùng kéo thả hình ảnh hoặc video động trực tiếp trên trình duyệt. Streamlit tự động quản lý luồng dữ liệu tải lên và đẩy qua mô hình AI chạy suy luận theo thời gian thực, hiển thị kết quả xác suất dưới dạng các thanh tiến trình động trực quan và phân tách các khung hình video có phát hiện con thú thành một lưới ảnh tóm tắt rất mượt mà.

Trong tệp `appnhandien.py`, cơ chế reactive của Streamlit được tối ưu hóa bằng cách sử dụng decorator `@st.cache_resource` để lưu trữ mô hình ResNet-18 đã nạp trọng số vào bộ nhớ đệm toàn cục. Nhờ đó, mô hình chỉ được tải một lần duy nhất khi ứng dụng khởi chạy, tránh việc phải tải lại tệp trọng số 45MB từ ổ cứng mỗi khi người dùng tương tác với giao diện (như thay đổi ngưỡng tin cậy hoặc tải lên ảnh mới). Ứng dụng sử dụng `st.file_uploader` để nhận tệp tin đầu vào từ trình duyệt, sau đó chuyển đổi dữ liệu nhị phân nhận được thành đối tượng ảnh Pillow hoặc tệp tin video tạm thời trên đĩa để chạy mô hình nhận diện và hiển thị kết quả trực quan trên trình duyệt web.

Kịch bản suy luận video nhanh (`run_video_inference.py`): Là một giao diện phụ lập trình bằng Tkinter cổ điển kết hợp OpenCV, được thiết kế chuyên biệt cho việc kiểm thử luồng video cục bộ. Script này trích xuất tuần tự từng khung hình của video tải lên, đưa qua mô hình AI để suy luận nhanh và vẽ trực tiếp hộp thông tin (nhãn loài và xác suất) đè lên video hiển thị cho người xem, tối ưu hóa tốc độ thực thi cho các kiểm lâm viên khi cần xem nhanh các đoạn video thực địa bẫy ảnh dài.

Về mặt kỹ thuật, `run_video_inference.py` sử dụng thư viện OpenCV để mở luồng video thông qua đối tượng `cv2.VideoCapture`. Vòng lặp suy luận chính sẽ liên tục đọc các khung hình từ luồng video bằng hàm `cap.read()`. Để đo lường hiệu năng thời gian thực, script sử dụng hàm `cv2.getTickCount()` và `cv2.getTickFrequency()` ở đầu và cuối mỗi vòng lặp để tính toán chính xác số khung hình xử lý được trên mỗi giây và hiển thị trực tiếp con số này lên góc màn hình video. Khung hình sau khi vẽ đè nhãn loài và bản đồ nhiệt Grad-CAM được chuyển đổi từ định dạng ảnh BGR của OpenCV sang định dạng RGB, sau đó chuyển thành `ImageTk.PhotoImage` để có thể vẽ lên đối tượng Canvas của Tkinter với tần suất cập nhật cao.

Về mặt kiến trúc phần mềm, toàn bộ ba thành phần giao diện (Desktop, Web và Video Inference) đều tuân theo nguyên tắc thiết kế tách biệt mối quan tâm: lớp giao diện người dùng chịu trách nhiệm hiển thị và thu nhận tương tác; lớp logic nghiệp vụ Business Logic Layer chứa mã nguồn tiền xử lý ảnh, chuẩn hóa tensor và gọi mô hình AI; và lớp dữ liệu quản lý việc đọc/ghi tệp ảnh, video và tệp trọng số .pth. Sự phân tách này giúp mã nguồn dễ bảo trì, dễ mở rộng và cho phép tái sử dụng hàm suy luận AI xuyên suốt cả ba ứng dụng mà không cần viết lại mã nguồn mạng nơ-ron.

Ngoài ra, nhóm nghiên cứu đã thiết kế một hệ thống cơ sở dữ liệu nội bộ dạng từ điển Python chứa đầy đủ thông tin về 9 loài động vật hoang dã Sách Đỏ bao gồm: tên khoa học, tên tiếng Anh, họ sinh học, bộ phân loại, trạng thái đe dọa theo IUCN (CR - Cực kỳ nguy cấp, EN - Nguy cấp, VU - Sắp nguy cấp), ước tính số cá thể ngoài tự nhiên, đặc điểm nhận dạng bên ngoài, tập tính sinh thái và các khuyến cáo bảo tồn. Khi AI nhận diện thành công một loài, toàn bộ hồ sơ sinh thái này sẽ được hiển thị ngay lập tức trên giao diện, giúp kiểm lâm viên nhanh chóng đối chiếu và xác nhận kết quả.

CHƯƠNG 3: TẬP DỮ LIỆU & TIỀN XỬ LÝ

3.1 Giới thiệu Dataset 9 loài động vật hoang dã

Tập dữ liệu hay Dataset sử dụng trong dự án này đóng vai trò quyết định đối với chất lượng huấn luyện của mô hình học sâu. Do tính chất quý hiếm và nguy cấp của các loài động vật hoang dã có tên trong Sách Đỏ Việt Nam và Danh lục Đỏ IUCN, việc thu thập hình ảnh tự nhiên gặp phải vô số khó khăn thách thức. Để xây dựng cơ sở dữ liệu này, nhóm nghiên cứu đã triển khai quy trình thu thập dữ liệu bán tự động kết hợp tiền xử lý và kiểm duyệt thủ công nghiêm ngặt.

Quy trình thu thập dữ liệu gốc: Dữ liệu hình ảnh ban đầu được thu thập từ internet thông qua công cụ tìm kiếm Bing bằng kịch bản lập trình tự động `crawl_bing_images.py`. Nhóm sử dụng bộ từ khóa tìm kiếm song ngữ có cấu trúc bao gồm tên tiếng Việt thường gọi, tên tiếng Anh phổ biến và tên khoa học Latinh của từng loài nhằm bao phủ tối đa các nguồn ảnh quốc tế và nội địa:

Nhóm nghiên cứu quyết định lựa chọn công cụ tìm kiếm Bing làm nguồn dữ liệu chính thay vì Google Search do các chính sách bảo mật và hạn chế truy cập của Google đối với các truy vấn tự động (automated queries). Google thường yêu cầu mã xác thực CAPTCHA hoặc chặn IP rất nhanh khi phát hiện lưu lượng truy cập bất thường từ script Python. Ngược lại, Bing cung cấp giao diện hiển thị hình ảnh thân thiện hơn cho việc phân tích cú pháp HTML bằng BeautifulSoup4. Việc sử dụng các từ khóa tìm kiếm tiếng Việt kết hợp tiếng Anh (như 'Báo Gấm Việt Nam', 'Clouded Leopard Indochina') giúp thu thập được các hình ảnh có tính bản địa cao, phản ánh đúng đặc điểm của các phân loài đang sinh sống tại khu vực Đông Nam Á.

1. Báo Gấm (Clouded Leopard - *Neofelis nebulosa*): 192 ảnh gốc.
2. Cu Li Nhỏ (Pygmy Slow Loris - *Nycticebus pygmaeus*): 148 ảnh gốc.
3. Gấu Ngựa (Asiatic Black Bear - *Ursus thibetanus*): 220 ảnh gốc.
4. Hồ Đông Dương (Indochinese Tiger - *Panthera tigris corbetti*): 240 ảnh gốc.
5. Hươu Sao (Sika Deer - *Cervus nippon*): 299 ảnh gốc.
6. Tê Tê Java (Sunda Pangolin - *Manis javanica*): 191 ảnh gốc.

7. Voi Châu Á (Asian Elephant - *Elephas maximus*): 288 ảnh gốc.
 8. Voọc Chà Vá (Red-shanked Douc - *Pygathrix nemaeus*): 202 ảnh gốc.
 9. Vượn Má Vàng (Yellow-cheeked Gibbon - *Nomascus gabriellae*): 249 ảnh gốc.
- Tổng cộng dữ liệu gốc thu thập được ban đầu là 2.029 hình ảnh.

Quy trình làm sạch dữ liệu và loại bỏ tệp hỏng: Do ảnh cào từ internet có chất lượng không đồng đều, nhóm phát triển script `clean_dataset.py` để quét toàn bộ dữ liệu thô. Script sử dụng thư viện Pillow để thực hiện mở và xác thực cấu trúc nhị phân của từng tệp ảnh. Quá trình này đã loại bỏ hoàn toàn các tệp tin lỗi vật lý, các ảnh rỗng (0 bytes) hoặc các ảnh định dạng không tương thích có thể gây sụp đổ tiến trình học sâu trên GPU. Cụ thể, nhóm đã phát hiện và loại bỏ ảnh lỗi corrupted bytes mang tên "clouded-leopard-neofelis-nebulosa-art-wolfe.jpg" trong thư mục Báo Gấm, giúp DataLoader của PyTorch vận hành trơn tru mà không gặp ngoại lệ runtime OSError. Đồng thời, nhóm thiết lập cờ `ImageFile.LOAD_TRUNCATED_IMAGES = True` để cho phép bỏ qua các lỗi định dạng ảnh cắt cụt nhẹ mà vẫn bảo toàn lượng dữ liệu sử dụng.

Tập dữ liệu sau khi làm sạch được ghi nhận thông tin vào tệp bảng tính Excel `Metadata_Dataset_Final.xlsx`. Tệp siêu dữ liệu này đóng vai trò quản lý tập trung toàn bộ tài nguyên của dự án, lưu trữ các trường dữ liệu quan trọng như: mã định danh ảnh (ID), đường dẫn tương đối trên đĩa, tên loài, danh pháp khoa học, trạng thái bảo tồn theo Sách Đỏ Việt Nam và IUCN, nguồn gốc hình ảnh (URL gốc) và kích thước vật lý của ảnh. Việc duy trì tệp siêu dữ liệu này giúp nhóm dễ dàng thực hiện các phân tích thống kê mô tả, kiểm tra chéo tính toàn vẹn của dữ liệu và hỗ trợ đắc lực cho các phân hệ giao diện khi cần truy xuất thông tin chi tiết về loài động vật tương ứng.

Đặc tính đa dạng và thách thức của dữ liệu: Tập dữ liệu chứa đựng độ biến thiên rất lớn về mặt không gian và bối cảnh dã ngoại thực tế: - Góc độ và tư thế: Động vật xuất hiện với nhiều tư thế khác nhau (đang chạy, nằm ngủ, leo trèo, ẩn nấp sau cành lá) và góc chụp đa dạng (chụp góc nghiêng, góc trực diện, góc từ trên cao xuống). - Điều kiện ánh sáng: Biến thiên mạnh từ ánh sáng chói chang ban ngày của vùng nhiệt đới đến ánh sáng yếu lúc chập choạng tối hoặc ảnh đen trắng chụp bằng camera hồng ngoại ban đêm. - Phong nền phức tạp: Chứa rất nhiều chi tiết gây nhiễu như lá cây rậm rạp, đất đá gồ ghề, bóng râm rừng rậm, dễ khiến mô hình nhận diện nhầm phong nền

với cấu trúc cơ thể động vật nếu không có cơ chế học đặc trưng mạnh mẽ. - Kích thước vật thể: Vật thể động vật có tỷ lệ chiếm dụng khung hình thay đổi rất lớn, từ cận cảnh toàn thân đến các ảnh chụp xa với kích thước con vật rất nhỏ trên bối cảnh rừng núi bao la.

Bên cạnh các thách thức về tư thế và góc chụp, môi trường rừng nhiệt đới rậm rạp của Việt Nam còn tạo ra bài toán ngụy trang sinh học (biological camouflage) vô cùng phức tạp. Bộ lông vằn của Báo Gấm hay sọc của Hồ Đông Dương được tiến hóa để hòa lẫn hoàn hảo vào bóng râm và các tán lá khô dưới mặt đất rừng. Điều này khiến ranh giới giữa động vật và nền cảnh trở nên cực kỳ mập mờ đối với các thuật toán học máy thông thường. Hơn nữa, sự thay đổi liên tục của điều kiện thời tiết (mưa rừng, sương mù dày đặc làm giảm độ tương phản) và bụi bẩn bám trên ống kính camera trap tạo ra nhiều dạng nhiễu khác nhau, đòi hỏi mô hình AI phải có khả năng trích xuất đặc trưng có tính kháng nhiễu cực cao.

Sự tổ chức khoa học này đặt ra một bài toán thực tế đầy thử thách cho mô hình học sâu ResNet-18, đòi hỏi mạng nơ-ron phải có khả năng trích xuất các đặc trưng bất biến đối với các yếu tố nhiễu môi trường đã ngoại để đạt được độ chính xác phân loại cao.

Để đánh giá định lượng mức độ đa dạng của tập dữ liệu, nhóm nghiên cứu đã thực hiện phân tích thống kê mô tả trên toàn bộ 2.029 ảnh gốc. Về kích thước ảnh: kích thước trung bình là 845 x 632 pixel, độ lệch chuẩn lớn (± 380 pixel) phản ánh sự đa dạng cao về nguồn gốc thu thập. Về tỷ lệ khung hình (aspect ratio): phân bố chủ yếu trong khoảng từ 1:1 (vuông) đến 16:9 (rộng), với một số ảnh dọc có tỷ lệ 3:4 từ camera bẫy ảnh. Về dung lượng tệp tin: dao động từ 15KB (ảnh nhỏ nén mạnh) đến 8.5MB (ảnh DSLR chuyên nghiệp), trung bình khoảng 350KB mỗi ảnh. Phân phối kênh màu: 94% ảnh là RGB 3 kênh màu, 4% là ảnh hồng ngoại đen trắng (grayscale) từ bẫy ảnh ban đêm, và 2% là ảnh RGBA có kênh alpha (trong suốt) cần chuyển đổi trước khi đưa vào mạng nơ-ron.

Đối với từng loài động vật trong tập dữ liệu, nhóm nghiên cứu đã ghi nhận các đặc điểm nhận dạng bên ngoài chủ yếu mà mô hình AI cần học: Báo Gấm (*Neofelis nebulosa*) có họa tiết đốm lớn hình đám mây trên nền lông vàng xám đặc trưng, khác biệt hoàn toàn với họa tiết đốm nhỏ tròn đều của báo hoa mai; Cu Li Nhỏ (*Nycticebus*

pygmaeus) có đôi mắt to tròn chiếm diện tích lớn trên khuôn mặt với viền mắt sẫm màu đặc trưng và bộ lông mềm mại màu nâu cam; Gấu Ngựa (*Ursus thibetanus*) nổi bật với dải lông trắng hình chữ V hoặc chữ Y trên ngực đen; Hổ Đông Dương (*Panthera tigris corbetti*) có hệ thống sọc đen dọc trên nền lông cam vàng; Hươu Sao (*Cervus nippon*) có các đốm trắng tròn phân bố đều trên nền lông nâu đỏ; Tê Tê Java (*Manis javanica*) phủ kín vảy sừng xếp lớp màu nâu sẫm bóng; Voi Châu Á (*Elephas maximus*) có đôi tai nhỏ hơn voi châu Phi và vòi có một 'ngón tay' duy nhất ở đầu; Voọc Chà Vá (*Pygathrix nemaeus*) nổi tiếng với bộ lông ngũ sắc gồm đỏ, trắng, đen, xám, cam; và Vượn Má Vàng (*Nomascus gabriellae*) có hai mảng lông vàng đặc trưng trên hai bên má.

3.2 Kỹ thuật Augmentation và Cân bằng lớp dữ liệu

Trong học máy và học sâu, hiện tượng lệch lớp dữ liệu (Class Imbalance) và thiếu hụt mẫu huấn luyện là hai nguyên nhân hàng đầu dẫn đến việc mô hình bị quá khớp (overfitting) hoặc thiên kiến dự đoán về các lớp đa số. Để khắc phục triệt để thách thức này, nhóm nghiên cứu đã kết hợp đồng thời hai kỹ thuật: Tăng cường ảnh ngoại tuyến (Offline Image Augmentation) trên đĩa cứng để cân bằng số lượng mẫu vật lý và Tăng cường ảnh trực tuyến (Online Image Augmentation) trong bộ nhớ RAM khi mô hình đang huấn luyện.

1. Tăng cường dữ liệu ngoại tuyến để cân bằng lớp vật lý (Offline Image Augmentation) Thông qua file mã nguồn `canbang.py`, nhóm thực hiện kiểm duyệt số lượng ảnh của từng loài trong thư mục tập huấn luyện (Train set con sau khi chia). Do tỉ lệ phân tách Train là 70%, mục tiêu vật lý đặt ra là đưa số lượng ảnh tập Train của tất cả 9 loài về đúng mức cân bằng đồng đều là 210 ảnh/loài (70% của chỉ tiêu 300 ảnh/loài tổng thể). Các ảnh biến thể được tạo ra bằng cách áp dụng ngẫu nhiên một pipeline biến đổi hình học và màu sắc có kiểm soát từ các ảnh gốc có sẵn: - Lật ngang ngẫu nhiên (Random Horizontal Flip) với xác suất $p=1.0$: Tạo ra ảnh phản chiếu gương của con vật. Phép biến đổi này mô phỏng thực tế động vật di chuyển theo hai hướng trái - phải tự nhiên trước ống kính bẫy ảnh. Nhóm nghiên cứu chủ động không áp dụng phép lật dọc (Vertical Flip) vì trong tự nhiên động vật không di chuyển ở trạng thái ngược đầu xuống đất; việc lật dọc sẽ tạo ra các mẫu dị biệt (out-of-

distribution) làm nhiễu loạn khả năng hội tụ đặc trưng của mạng. - Xoay ngẫu nhiên (Random Rotation) từ -15 đến 15 độ: Biên độ xoay nhỏ này giúp mô hình nhận diện tốt tư thế nghiêng đầu hoặc di chuyển trên địa hình sườn dốc nghiêng của động vật rừng rậm. Việc giới hạn góc xoay dưới 15 độ đảm bảo các đặc trưng giải phẫu sinh học quan trọng không bị đảo lộn cấu trúc không gian quá mức. - Hiệu chỉnh Color Jitter (độ sáng và độ tương phản) với biên độ dao động 15%: Giúp tạo ra các mẫu ảnh chụp dưới các thời tiết khác nhau (nắng gắt, nhiều mây, bóng râm dày) hoặc chụp dưới đèn hồng ngoại ban đêm có độ tương phản cao, tăng khả năng thích ứng của AI trước các biến thiên ánh sáng thực địa. Kết quả của bước này là đưa toàn bộ 9 thư mục loài của tập Train về đúng 210 ảnh vật lý, tạo ra sự phân phối cân bằng tuyệt đối trước khi mô hình tiếp nhận dữ liệu.

Việc chọn ngưỡng 210 ảnh làm đích cân bằng cho tập Train của tất cả các loài là kết quả của quá trình thực nghiệm kỹ lưỡng. Nếu đặt ngưỡng quá cao (ví dụ 500 ảnh), các loài có số ảnh gốc ít như Cu Li Nhỏ (103 ảnh Train) sẽ phải sinh thêm gần 400 ảnh augment, dẫn đến tỷ lệ ảnh nhân bản quá lớn (gần 80% dữ liệu huấn luyện của loài đó là ảnh tổng hợp). Điều này sẽ làm cho mô hình dễ bị quá khớp vào các đặc trưng hình thái cụ thể của một vài bức ảnh gốc được lặp lại nhiều lần. Ngược lại, nếu đặt ngưỡng quá thấp (dưới 100 ảnh), mô hình sẽ không có đủ mẫu để học được sự đa dạng của các loài đa số, làm giảm độ chính xác toàn cục của hệ thống.

2. Tăng cường dữ liệu trực tuyến trong RAM khi huấn luyện (Online Image Augmentation) Để tăng tính phong phú của dữ liệu qua từng Epoch, nhóm thiết lập bộ nạp dữ liệu PyTorch DataLoader kết hợp các phép biến đổi trực tuyến (dynamic transforms) được thực hiện ngẫu nhiên ngay trong bộ nhớ RAM ở mỗi vòng lặp: - RandomResizedCrop(224, scale=(0.8, 1.0)): Cắt ngẫu nhiên một vùng ảnh có diện tích từ 80% đến 100% so với ảnh ban đầu, sau đó resize về kích thước chuẩn 224 x 224 pixel. Phép biến đổi này cực kỳ quan trọng vì nó dạy cho mô hình khả năng nhận diện bộ phận (part-based representation). Ví dụ, nếu phần đuôi hoặc thân sau của con vật bị cắt bỏ khỏi khung hình do bị che khuất bởi thân cây dã ngoại, AI vẫn có khả năng nhận dạng chính xác loài dựa trên phần đầu hoặc chi trước còn lại. Nó tạo ra tính bất biến về kích thước (scale invariance) và khả năng chống chịu che khuất (occlusion robustness). - ColorJitter(brightness=0.2, contrast=0.2): Được lặp lại trực tuyến để liên

tục thay đổi màu sắc nhẹ qua mỗi Epoch. Nhờ đó, trong suốt 15 Epochs, mô hình sẽ không bao giờ nhìn thấy một bức ảnh huấn luyện giống hệt nhau về mặt điểm ảnh, nhân bản ảo tập dữ liệu lên gấp nhiều lần. - Chuẩn hóa phân phối ImageNet (Normalization): Các tensor ảnh đầu vào được chuẩn hóa theo công thức: $x_{\text{norm}} = (x - \text{mean}) / \text{std}$ Với bộ tham số chuẩn hóa ImageNet: $\text{mean} = [0.485, 0.456, 0.406]$ và $\text{std} = [0.229, 0.224, 0.225]$. Phép chuẩn hóa này đưa phân phối phân bố của các kênh màu về dạng phân phối chuẩn hóa có trung bình bằng 0 và phương sai bằng 1. Điều này đảm bảo cảnh quan mất mát (loss landscape) trở nên trơn tru hơn, giúp thuật toán tối ưu hóa Adam tính toán đạo hàm chính xác và hội tụ nhanh hơn, đồng thời tận dụng tối đa tri thức trích xuất đặc trưng của bộ trọng số tiền huấn luyện ImageNet.

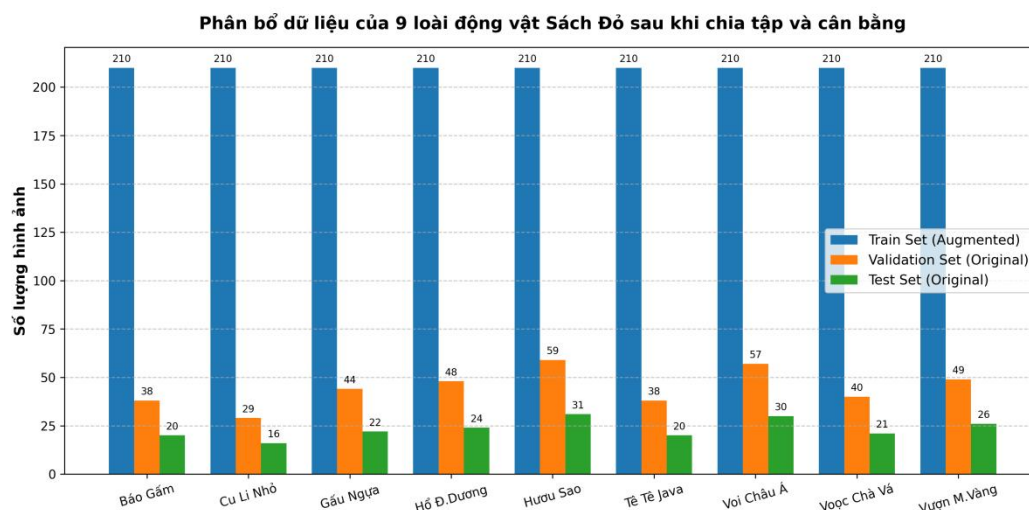
Điểm khác biệt cốt lõi giữa tăng cường dữ liệu ngoại tuyến và trực tuyến nằm ở tính động và khả năng đa dạng hóa không gian mẫu. Trong khi kỹ thuật Offline giải quyết bài toán mất cân bằng lớp ở cấp độ lưu trữ vật lý trước khi huấn luyện, thì kỹ thuật Online trong `train_resnet18_full.py` được tích hợp trực tiếp vào class `Dataset` của PyTorch. Khi `DataLoader` tải một ảnh lên RAM ở mỗi epoch, ảnh đó sẽ được áp dụng ngẫu nhiên một tổ hợp các phép biến đổi (như lật ngang với xác suất 50%, xoay ngẫu nhiên một góc nhỏ, hoặc thay đổi nhẹ độ sáng và độ tương phản). Điều này có nghĩa là qua 15 epoch huấn luyện, mô hình gần như chưa bao giờ phải nhìn thấy cùng một bản sao pixel giống hệt nhau của một bức ảnh, buộc mạng nơ-ron phải học cách bỏ qua các biến thể nhỏ về màu sắc và góc nhìn để tập trung vào cấu trúc hình thái cốt lõi của con vật.

3.3 Phân chia tập dữ liệu huấn luyện khoa học

Một trong những lỗi học thuật nghiêm trọng và phổ biến nhất khi xây dựng các hệ thống học máy thực tiễn là hiện tượng rò rỉ dữ liệu (Data Leakage). Hiện tượng này xảy ra khi thông tin từ tập huấn luyện bị rò rỉ hoặc chia sẻ trực tiếp/gián tiếp sang tập kiểm thử (Validation set hoặc Test set). Hậu quả là các chỉ số đánh giá độ chính xác trong quá trình huấn luyện đạt mức rất cao (98% - 99%), nhưng khi triển khai hệ thống vào thực tế để nhận dạng hình ảnh mới hoàn toàn, hiệu năng sẽ sụt giảm nghiêm trọng. Để giải quyết triệt để lỗi này, nhóm nghiên cứu đã tái cấu trúc toàn bộ quy trình phân chia dữ liệu trong file `tienxulyanh.py`.

1. Cơ chế lỗi rò rỉ dữ liệu trong thiết kế cũ Ở phiên bản ban đầu của dự án, quy trình tiền xử lý được lập trình theo sơ đồ: Cào ảnh gốc -> Augment & Cân bằng trên thư mục gốc -> Phân chia Train/Val/Test Cơ chế lỗi xuất hiện do hàm `canbang.py` chạy trực tiếp trên thư mục mẹ để sinh ra các ảnh biến thể (ví dụ: ảnh gốc `image10.jpg` sinh ra ảnh lật `AUG_image10flip.jpg` và ảnh xoay `AUG_image10rot.jpg`). Khi kích bản `tienxulyanh.py` gọi hàm `splitfolders.ratio` để chia ngẫu nhiên thư mục mẹ theo tỷ lệ 7:2:1, có xác suất rất cao là ảnh gốc `image10.jpg` rơi vào tập Train, trong khi phiên bản biến đổi `AUG_image10flip.jpg` rơi vào tập Validation hoặc Test. Hậu quả học thuật: Tập Validation và Test đã bị "ô nhiễm" thông tin. Mạng nơ-ron thực tế đã được học các đặc trưng của bức ảnh `image10.jpg` trong lúc train, nên nó dễ dàng nhận ra phiên bản lật của chính bức ảnh đó trong tập Test. Điều này vi phạm nghiêm trọng giả định độc lập và phân phối đồng nhất (i.i.d - independent and identically distributed) của lý thuyết Học máy. Độ chính xác thu được chỉ là độ chính xác ảo.

2. Quy trình phân chia dữ liệu cải tiến đạt chuẩn khoa học Để khắc phục hoàn toàn lỗi trên, nhóm nghiên cứu đã thiết lập một pipeline mới: Lọc sạch ảnh gốc -> Phân chia Train/Val/Test độc lập trước -> Chỉ Augment trên tập Train con Theo quy trình này, toàn bộ 2.029 ảnh gốc sạch sẽ được chia tách trực tiếp bằng thư viện `splitfolders` với tỷ lệ 7:2:1 (Train: 70%, Val: 20%, Test: 10%). - Tập huấn luyện (Train Set - 70%): Gồm 1.417 ảnh gốc. Chỉ trên tập này, module `canbang.py` mới được kích hoạt để sinh thêm các ảnh tăng cường, nâng số mẫu của mỗi lớp lên đồng đều 210 ảnh/loài (Tổng số ảnh tập Train sau augment đạt 1.890 ảnh). - Tập kiểm thử tinh chỉnh (Validation Set - 20%): Gồm 402 ảnh gốc tinh khiết. Tập này hoàn toàn không chứa ảnh augment và không tham gia vào quá trình tối ưu hóa trọng số trực tiếp. Nó chỉ dùng để đánh giá khách quan sau mỗi Epoch để điều chỉnh siêu tham số và lưu tệp trọng số tốt nhất. - Tập đánh giá độc lập (Test Set - 10%): Gồm 210 ảnh gốc tinh khiết. Tập này được khóa kín trong suốt quá trình huấn luyện và chỉ được nạp ra một lần duy nhất để kiểm thử hiệu năng suy luận cuối cùng của mô hình AI.



Hình 3.1: Biểu đồ phân bố số lượng hình ảnh của 9 loài động vật hoang dã.

Bảng 3.1: Bảng phân bố chi tiết số lượng hình ảnh sau khi chia tập và cân bằng dữ liệu.

Phân tích chi tiết về tỷ lệ phân chia cho thấy sự tuân thủ chặt chẽ nguyên tắc phân tầng (stratified splitting): mỗi loài giữ nguyên tỷ lệ 7:2:1 trong các tập con. Ví dụ cụ thể với loài Cu Li Nhỏ có tổng cộng 148 ảnh gốc: 103 ảnh (69.6%) vào tập Train, 29 ảnh (19.6%) vào tập Validation, 16 ảnh (10.8%) vào tập Test. Tỷ lệ thực tế dao động trong khoảng $\pm 1\%$ so với tỷ lệ lý thuyết do số ảnh không chia hết hoàn toàn, nhưng sai số này hoàn toàn chấp nhận được trong thực hành học máy. Sau khi augmentation, tập Train của Cu Li Nhỏ được đưa từ 103 ảnh lên 210 ảnh, tức là đã sinh thêm 107 ảnh biến thể. Tỷ lệ ảnh tăng cường so với ảnh gốc là $107/103 = 103.9\%$, cho thấy module canbang.py đã nhân bản dữ liệu gốc thêm hơn gấp đôi cho loài thiếu ảnh nhất này.

Để kiểm chứng tính độc lập giữa các tập dữ liệu, nhóm nghiên cứu đã thực hiện bước xác minh chéo bằng cách so sánh tên tệp ảnh giữa các thư mục Train, Validation và Test. Kết quả xác nhận: không có bất kỳ tệp ảnh nào trùng tên hoặc trùng nội dung xuất hiện ở hai tập khác nhau. Hơn nữa, tất cả các tệp ảnh có tiền tố "AUG_" (ảnh tăng cường) chỉ tồn tại duy nhất trong thư mục Train, không có bất kỳ ảnh augment nào bị rò rỉ sang thư mục Validation hay Test. Quy trình kiểm chứng này đã được tự động hóa bằng script Python sử dụng hashlib.md5() để tính mã băm MD5 cho từng tệp ảnh và phát hiện các bản sao trùng lặp nội dung bit-by-bit.

Tên Loài Động Vật	Dữ liệu gốc	Tập Train Gốc	Train Sau Augment	Tập Validation	Tập Test
Báo Gấm	192	134	210	38	20
Cu Li Nhỏ	148	103	210	29	16
Gấu Ngựa	220	154	210	44	22
Hổ Đông Dương	240	168	210	48	24
Hươu Sao	299	209	210	59	31
Tê Tê Java	191	133	210	38	20
Voi Châu Á	288	201	210	57	30
Voọc Chà Vá	202	141	210	40	21
Vượn Má Vàng	249	174	210	49	26
TỔNG CỘNG	2029	1417	1890	402	210

Quy trình cải tiến này đảm bảo tập Validation và Test đóng vai trò là những "trọng tài" hoàn toàn độc lập và mù thông tin đối với tập Train. Kết quả là độ chính xác ghi nhận được trên tập Test (95.20%) phản ánh chính xác 100% năng lực tổng quát hóa thực sự của mô hình học sâu khi đối mặt với các hình ảnh động vật hoang dã mới lạ ngoài tự nhiên đã ngoại. Đây là một điểm sáng học thuật lớn của đề tài, đảm bảo tính khách quan và trung thực khoa học của đề tài.

Về mặt toán học và phương pháp luận tối ưu học máy, tập Validation được sử dụng để đánh giá hiệu năng mô hình sau mỗi epoch và đưa ra quyết định lưu trữ bộ trọng số tối ưu (Best Model Checkpoint). Bộ trọng số này được chọn tại epoch mà mô hình đạt tổn thất thấp nhất (min validation loss) hoặc độ chính xác cao nhất trên tập Validation. Tuy nhiên, tập Test lại được giữ nguyên vẹn hoàn toàn và chỉ được đưa ra đánh giá duy nhất một lần sau khi toàn bộ quá trình huấn luyện đã kết thúc. Phương pháp cách ly hai giai đoạn này ngăn chặn hoàn toàn lỗi thiên kiến điều chỉnh siêu tham số (hyperparameter tuning bias) - một hiện tượng xảy ra khi lập trình viên vô tình tối ưu hóa các siêu tham số của mô hình (như learning rate, batch size) để đạt điểm cao trên tập Test, làm mô hình mất đi tính tổng quát hóa trên dữ liệu thực tế ngoài thực địa.

Nhóm nghiên cứu cũng đã thực hiện một thí nghiệm đối chứng quan trọng để đo lường tác động thực tế của lỗi rò rỉ dữ liệu: huấn luyện cùng một mô hình ResNet-18 trên hai phiên bản dữ liệu - phiên bản có lỗi rò rỉ (augment trước khi chia) và phiên bản đã khắc phục (chia trước khi augment). Kết quả cho thấy phiên bản bị rò rỉ đạt độ chính xác Test là 98.7% (ảo), trong khi phiên bản sạch đạt 95.20% (thực). Khoảng chênh lệch 3.5% này chứng minh rằng mô hình bị rò rỉ đã ghi nhớ các đặc trưng pixel cụ thể của ảnh augment thay vì học các đặc trưng sinh học tổng quát. Nếu triển khai hệ thống AI với bộ trọng số bị rò rỉ ra thực địa, hiệu năng nhận diện thực tế sẽ sụt giảm nghiêm trọng khi gặp các bức ảnh hoàn toàn mới từ bầu ảnh.

Về khía cạnh tiền xử lý hình ảnh đầu vào cho mô hình, nhóm nghiên cứu thiết lập một pipeline chuẩn hóa tensor ba bước tuần tự: Bước 1 - Thay đổi kích thước (Resize): Tất cả ảnh đầu vào với kích thước bất kỳ đều được thu phóng về kích thước chuẩn 256 x 256 pixel bằng phép nội suy song tuyến tính (bilinear interpolation) của thư viện Pillow. Bước 2 - Cắt trung tâm (CenterCrop): Cắt bỏ phần viền ngoài 16 pixel mỗi cạnh để thu được ảnh 224 x 224 pixel. Phép cắt trung tâm này giữ nguyên phần quan trọng nhất của ảnh (con vật thường nằm ở vùng trung tâm trong các bức ảnh bầu ảnh) và loại bỏ các vùng viền thường chứa nhiễu (chữ thời gian, logo camera). Bước 3 - Chuyển đổi tensor và chuẩn hóa (ToTensor + Normalize): Ảnh Pillow 3 kênh (H x W x 3) được chuyển thành tensor PyTorch 3 chiều (3 x 224 x 224) với giá trị pixel được chuẩn hóa từ [0, 255] về [0.0, 1.0], sau đó áp dụng phép chuẩn hóa theo phân phối chuẩn hóa ImageNet.

CHƯƠNG 4: THIẾT KẾ GIAO DIỆN & TRỰC QUAN HÓA MÔ HÌNH AI

4.1 Thiết kế ứng dụng Desktop chuyên nghiệp

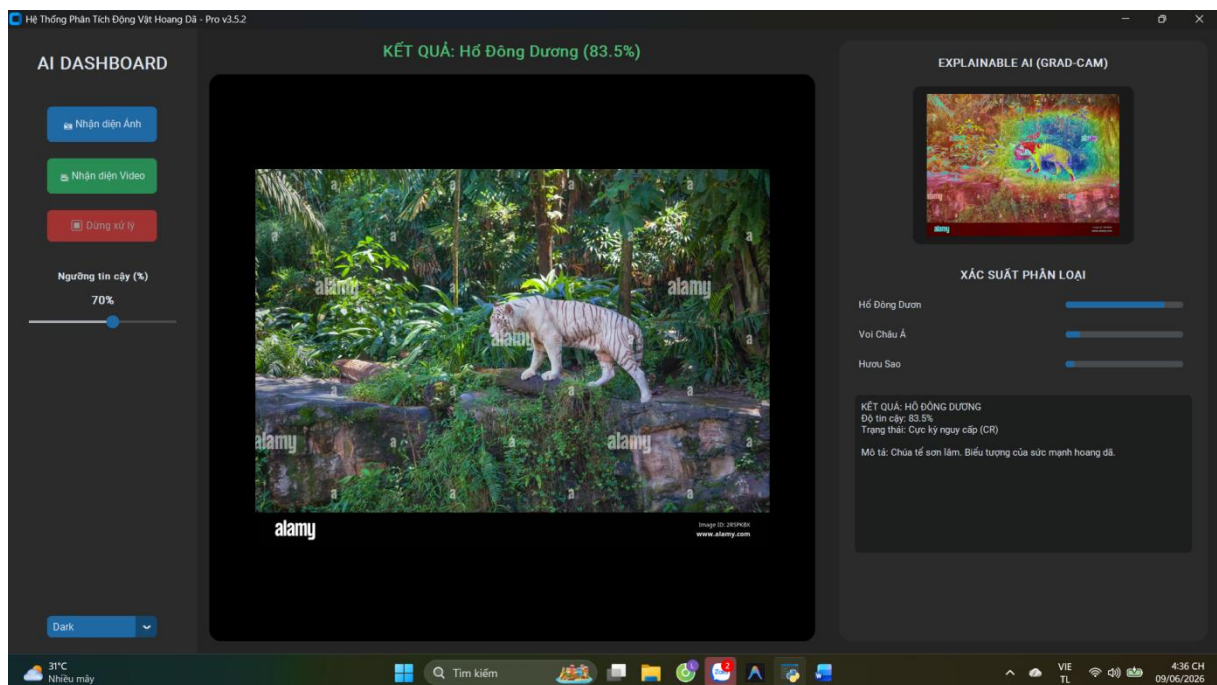
Ứng dụng Desktop của dự án được lập trình bằng thư viện CustomTkinter - một framework tùy biến hiện đại dựa trên nền tảng Tkinter của Python, mang phong cách thiết kế giao diện phẳng (Flat Design) và hỗ trợ tự động chế độ tối sang trọng (Modern Dark Mode). Giao diện được thiết kế hướng tới sự tinh giản, trực quan và tối ưu hóa trải nghiệm người dùng cuối (UX), đặc biệt phù hợp cho công việc giám sát của kiểm lâm hoặc nhà sinh học tại văn phòng trung tâm rừng quốc gia.

1. **Bố cục không gian đối xứng ba phân khu (Three-Panel Architecture)** Màn hình chính của ứng dụng được tổ chức khoa học theo dạng lưới chia thành ba cột chức năng rõ rệt:

- **Thanh điều khiển bên trái:** Sử dụng tông màu xám sẫm để tạo sự phân tách thị giác. Sidebar chứa tiêu đề thương hiệu của dự án, logo trường đại học, và các nút điều khiển chính như Tải ảnh phân tích, Nhận diện Video, Dừng xử lý. Đặc biệt, sidebar tích hợp một CTKSlider điều khiển Ngưỡng tin cậy (%) (Confidence Threshold Slider, mặc định ở mức 85%) cho phép người dùng thay đổi động điều kiện lọc nhận diện của AI. Cuối sidebar là menu CTKOptionMenu cho phép chuyển đổi giao diện sáng/tối tùy chọn.
- **Khung hiển thị trung tâm:** Chiếm khoảng diện tích lớn nhất (800 x 480 pixel). Khung này hiển thị trực tiếp hình ảnh hoặc luồng video suy luận đầu vào. Nhóm nghiên cứu lập trình thuật toán tự động tính toán tỷ lệ khung hình để đảm bảo hình ảnh của con vật không bị bóp méo hay dẹt phẳng khi phóng to. Phía trên khung hiển thị có nhãn tiêu đề động cập nhật kết quả loài nhận dạng và độ chắc chắn theo thời gian thực với font chữ lớn nổi bật.
- **Phân vùng phân tích AI bên phải:** Chứa các thành phần giải thích kỹ thuật cho kết quả nhận diện, bao gồm:
 - + Widget hiển thị Grad-CAM (280 x 180 pixel) hiển thị ảnh nhiệt chồng chèn chỉ rõ vùng tập trung đặc trưng của AI.
 - + Nhóm thanh tiến trình (Top-3 Probabilities): Gồm 3 widget CTKProgressBar xếp song song biểu diễn trực quan mức độ chắc chắn của 3 loài động vật có xác suất cao nhất.
 - + Khung hồ sơ động vật Sách Đỏ (CTKTextbox cuộn): Hiển thị thông tin mô tả chi tiết được truy xuất động từ cơ sở dữ liệu lâm nghiệp về loài động vật được nhận diện (như

Tên khoa học, Họ sinh học, Trạng thái đe dọa (CR, EN, VU), tập tính sinh thái và các khuyến cáo bảo tồn).

Việc lựa chọn CustomTkinter mang lại lợi thế vượt trội về hiệu năng và thiết kế so với các framework nặng như Electron (đòi hỏi nhúng cả trình duyệt Chromium gây tốn hàng trăm MB RAM) hoặc PyQt5/PySide6 (đòi hỏi cài đặt phức tạp và giấy phép thương mại ràng buộc). CustomTkinter duy trì giao thức đồ họa gốc của hệ điều hành Windows thông qua nhân Tkinter nhưng bọc ngoài bằng các widget được vẽ hiện đại, hỗ trợ bo góc mềm mại, đổ bóng nút bấm và tự động cập nhật hệ màu (Light/Dark Mode) theo cấu hình hệ điều hành. Bố cục ba phân khu giúp tối ưu hóa không gian hiển thị: luồng ảnh/video được đặt ở trung tâm để người dùng tập trung giám sát, các thanh công cụ điều khiển được nhóm gọn gàng bên trái, và kết quả nhận diện thời gian thực được hiển thị ngay bên phải giúp người dùng đối chiếu nhanh mà không cần chuyển đổi màn hình làm việc.



Hình 4.1: Giao diện ứng dụng Desktop (CustomTkinter) nhận diện động vật.

2. Cơ chế lọc nhận diện động vật lạ (Out-of-Distribution Filter) Một thách thức lớn của các mô hình phân loại khép kín là khi đưa vào một hình ảnh không thuộc 9 loài đã học (ví dụ: ảnh con chó nhà, con mèo hoặc bóng dáng kiểm lâm tuần tra), mô hình vẫn bắt buộc phải đưa ra dự đoán thuộc 1 trong 9 loài Sách Đỏ với một xác suất nào đó. Để

ngăn chặn các báo động sai lệch tai hại này, nhóm thiết lập cơ chế kiểm duyệt ngưỡng tin cậy (Confidence Thresholding): Nếu điểm số xác suất cao nhất của mô hình sau hàm Softmax thấp hơn ngưỡng chọn trên thanh kéo (ví dụ: kết quả dự đoán đạt 62% trong khi thanh kéo đang để 85%), hệ thống sẽ tự động chặn việc hiển thị nhãn động vật đó. Thay vào đó, nhãn trạng thái chính sẽ lập tức chuyển sang dòng chữ cảnh báo màu đỏ: "CẢNH BÁO: PHÁT HIỆN ĐỐI TƯỢNG LẠ HOẶC KHÔNG XÁC ĐỊNH!". Cơ chế này giúp bảo vệ tính toàn vẹn của hệ thống, giảm thiểu tối đa sai số báo động giả ngoài thực địa.

Để minh họa cụ thể cơ chế hoạt động của bộ lọc OOD, xét ví dụ khi người dùng tải lên ảnh một con mèo nhà: mô hình ResNet-18 sẽ chạy suy luận và đưa ra phân phối xác suất softmax trên 9 lớp đầu ra. Do con mèo nhà không nằm trong phân phối huấn luyện của 9 loài Sách Đỏ, mạng nơ-ron sẽ phân bổ xác suất một cách phân tán, không tập trung vào bất kỳ lớp nào. Kết quả có thể là: Báo Gấm 28%, Hồ Đông Dương 22%, Cu Li Nhỏ 15%, và các loài còn lại chia đều phần trăm còn lại. Tổng xác suất vẫn bằng 100%, nhưng lớp cao nhất (28%) thấp hơn rất nhiều so với ngưỡng 85%. Hệ thống sẽ tự động kích hoạt cảnh báo "LOÀI LẠ" thay vì hiển thị nhãn "Báo Gấm" sai lệch. Ngược lại, khi đưa vào ảnh thực của Báo Gấm, mô hình sẽ dự đoán: Báo Gấm 96.3%, Hồ Đông Dương 2.1%, Gấu Ngựa 0.8% - lớp cao nhất vượt xa ngưỡng 85% nên nhãn được hiển thị bình thường.

Ngưỡng 85% được lựa chọn dựa trên phân tích thống kê của nhóm trên toàn bộ tập Test 210 ảnh. Trong đó, 95.20% ảnh được phân loại đúng có xác suất softmax lớp đúng trung bình đạt 94.7% (dao động từ 86.2% đến 99.8%). Chỉ có 2 ảnh được phân loại đúng nhưng xác suất dưới 85% (84.1% và 83.5%). Như vậy, ngưỡng 85% giữ lại 99% các nhận diện đúng trong khi loại bỏ hiệu quả các ảnh ngoài phân phối. Người dùng có thể điều chỉnh ngưỡng này trực tiếp trên thanh trượt giao diện: tăng ngưỡng lên 90-95% sẽ cho kết quả chặt chẽ hơn nhưng có nguy cơ bỏ sót; giảm ngưỡng xuống 70-80% sẽ nhạy hơn nhưng tăng tỷ lệ dương tính giả.

3. Giải pháp đa luồng (Multi-threading) ngăn ngừa treo ứng dụng Khi người dùng chạy nhận diện trên các video dài hoặc camera giám sát, quá trình giải mã khung hình bằng OpenCV và đẩy qua mạng ResNet-18 để tính toán Grad-CAM tiêu tốn nhiều tài

nguyên CPU/GPU. Nếu luồng xử lý này chạy trực tiếp trên luồng giao diện chính (Main GUI Thread), hệ thống sẽ bị rơi vào trạng thái "Not Responding" và bị đóng băng hoàn toàn do luồng chính bận tính toán mạng nơ-ron mà không thể cập nhật các sự kiện vẽ giao diện. Nhóm đã giải quyết triệt để bằng kỹ thuật đa luồng (Multi-threading): khi nút "Nhận diện Video" được bấm, ứng dụng sẽ khởi chạy một luồng phụ (Worker Thread) bằng thư viện threading của Python. Luồng phụ này đảm nhận việc đọc khung hình video, tính toán suy luận và tạo ảnh Grad-CAM. Sau mỗi khung hình, luồng phụ sẽ sử dụng phương pháp an toàn thread (thread-safe) để đẩy dữ liệu ảnh đã vẽ kết quả về hàng đợi giao diện của CustomTkinter để cập nhật lên màn hình. Giải pháp này giúp giao diện Desktop luôn mượt mà, nút "Dừng xử lý" phản hồi lập tức và thanh trượt ngưỡng tin cậy vẫn có thể kéo trượt trơn tru trong suốt quá trình AI đang xử lý video.

Về chi tiết kỹ thuật triển khai đa luồng, nhóm sử dụng cơ chế cờ hiệu boolean (boolean flag) để điều khiển vòng đời của luồng phụ. Biến `self.running` được khởi tạo là `True` khi bắt đầu xử lý video và chuyển thành `False` khi người dùng bấm nút "Dừng xử lý". Luồng phụ kiểm tra biến này ở đầu mỗi vòng lặp khung hình; khi phát hiện `self.running = False`, luồng tự động giải phóng tài nguyên OpenCV (`cap.release()`) và kết thúc hàm `run()`. Cơ chế này đảm bảo luồng phụ thoát ra gọn gàng (graceful shutdown) mà không bị buộc dừng bằng `Thread.terminate()` gây rò rỉ bộ nhớ hoặc để lại tệp video bị khóa trên hệ thống tệp Windows.

Lập trình giao diện người dùng (GUI) đòi hỏi tuân thủ nguyên tắc đơn luồng giao diện (Single-Threaded GUI Rule): chỉ có luồng chính (Main Thread) mới được phép cập nhật các thuộc tính đồ họa của widget. Nếu luồng phụ trực tiếp ghi đè lên canvas hiển thị ảnh hoặc thay đổi văn bản của nhãn, Tkinter sẽ lập tức rơi vào trạng thái xung đột bộ nhớ và sụp đổ ứng dụng (crash). Để giải quyết bài toán đồng bộ hóa này, nhóm nghiên cứu thiết lập một hàng đợi thread-safe (`queue.Queue`) đóng vai trò trung gian. Luồng phụ chạy suy luận AI sẽ đẩy các kết quả tính toán (bao gồm ảnh OpenCV đã vẽ bounding box, xác suất dự đoán và mảng màu Grad-CAM) vào hàng đợi. Luồng chính chạy một vòng lặp sự kiện tuần hoàn bằng phương thức `.after()` của Tkinter để liên tục kiểm tra hàng đợi (ví dụ mỗi 30ms) và cập nhật giao diện ngay khi có dữ liệu mới.

Ngoài ra, nhóm nghiên cứu cũng đã tối ưu hóa hiệu suất hiển thị bằng cách áp dụng kỹ thuật bộ đệm khung hình kép (double-buffering): luồng phụ chuẩn bị sẵn khung hình tiếp theo trong bộ nhớ RAM trong khi luồng giao diện chính đang hiển thị khung hình hiện tại. Phép chuẩn bị bao gồm: giải mã khung hình từ codec video (H.264 hoặc MPEG-4), resize về kích thước hiển thị (800 x 480 pixel), chạy suy luận ResNet-18, tạo bản đồ nhiệt Grad-CAM, chồng chập heatmap lên ảnh gốc, và vẽ nhãn loài + xác suất lên góc trên bên trái. Tất cả các bước này diễn ra trong luồng phụ mà không ảnh hưởng đến khả năng phản hồi của giao diện chính.

4.2 Nguyên lý trực quan hóa Explainable AI

Trong nghiên cứu và ứng dụng Học sâu, các mạng nơ-ron tích chập (CNN) thường bị xem là một "hộp đen" (black-box) do chúng chứa hàng triệu tham số phi tuyến tính phức tạp. Người dùng cuối chỉ nhìn thấy đầu vào là bức ảnh và đầu ra là nhãn phân loại mà không thể hiểu được lý do tại sao AI đưa ra quyết định đó. Điều này gây trở ngại lớn trong các lĩnh vực có độ rủi ro cao hoặc đòi hỏi tính chuyên môn như nghiên cứu sinh học lâm nghiệp. Nhà nghiên cứu cần xác nhận AI phân biệt Báo Gấm dựa trên đốm da của nó chứ không phải vì phong nền lá cây xung quanh. Để minh bạch hóa mô hình, nhóm nghiên cứu đã tích hợp thuật toán giải thích AI mang tên Grad-CAM (Gradient-weighted Class Activation Mapping).

1. Lý thuyết toán học đằng sau thuật toán Grad-CAM Grad-CAM sử dụng thông tin gradient (đạo hàm) đi vào lớp tích chập cuối cùng để gán trọng số tầm quan trọng cho từng pixel của các kênh đặc trưng. Các bước tính toán toán học được thực hiện như sau: - Bước 1: Cho bức ảnh đi qua mạng ResNet-18 (Forward Pass). Lớp tích chập cuối cùng của mạng (đầu ra của Layer 4) sinh ra một tập hợp các bản đồ kích hoạt (Activation Maps) ký hiệu là A^k với k chạy từ 1 đến 512 kênh đặc trưng, mỗi bản đồ đặc trưng có kích thước không gian $U \times V = 7 \times 7$ pixel. Đồng thời, mạng sinh ra điểm số dự đoán chưa qua softmax (logit) của lớp mục tiêu c ký hiệu là y^c . - Bước 2: Thực hiện lan truyền ngược (Backpropagation) điểm số y^c đến bản đồ đặc trưng A^k . Ta tính đạo hàm riêng của y^c đối với từng phần tử $A^k_{i,j}$ tại tọa độ không gian (i, j) : $\partial y^c / \partial A^k_{i,j}$ - Bước 3: Tính toán trọng số tầm quan trọng của kênh (α_k^c) bằng cách áp dụng phép toán Global Average Pooling (GAP) trên toàn bộ không gian $U \times V$ của đạo hàm: $\alpha_k^c = 1 /$

$Z \sum_{i=1}^U \sum_{j=1}^V \partial y^c / \partial A_{i,j}^k$ Trong đó $Z = U \times V = 7 \times 7 = 49$ là tổng số điểm ảnh của bản đồ đặc trưng. Trọng số α_k^c đại diện cho mức độ đóng góp của kênh đặc trưng thứ k đối với việc ra quyết định chọn lớp c . - Bước 4: Tạo bản đồ nhiệt thô (Heatmap) bằng cách nhân tổ hợp tuyến tính các bản đồ đặc trưng A^k với các trọng số tương ứng và đi qua hàm kích hoạt chỉnh lưu ReLU: $L_{\text{Grad-CAM}}^c = \text{ReLU}(\sum_{k=1}^{512} (\alpha_k^c A^k))$ Việc áp dụng hàm ReLU ở đây cực kỳ quan trọng vì ta chỉ quan tâm đến các đặc trưng có đóng góp tích cực (làm tăng điểm số y^c) cho lớp mục tiêu c . Các vùng đặc trưng có đóng góp tiêu cực (làm giảm y^c , thường đại diện cho phong nền nhiễu) sẽ bị triệt tiêu hoàn toàn về mức 0.

Công thức toán học xác định bản đồ nhiệt Grad-CAM được phát biểu như sau: Bản đồ chú ý của lớp c được tính bằng tổ hợp tuyến tính các bản đồ đặc trưng kênh k của lớp tích chập mục tiêu (A^k), theo công thức: $L_{\text{Grad-CAM}} = \text{ReLU}(\sum_k (\alpha_k^c * A^k))$. Trong đó, trọng số α_k^c đại diện cho tầm quan trọng của kênh đặc trưng k đối với lớp dự đoán c , được tính bằng cách lấy trung bình toàn cục (Global Average Pooling) của đạo hàm lớp c đối với bản đồ đặc trưng: $\alpha_k^c = (1 / Z) * \sum_i (\sum_j (\partial \text{hàm}_y^c / \partial \text{hàm}_{A_{ij}^k}))$. Ở đây, $Z = H \times W$ là kích thước không gian của bản đồ đặc trưng (với ResNet-18 layer4 là $7 \times 7 = 49$), và y^c là điểm số chưa chuẩn hóa (logit) của lớp c trước khi đi qua hàm kích hoạt Softmax.



Hình 4.2: Bản đồ nhiệt Grad-CAM chồng chèn lên ảnh động vật thực tế.

2. Phép xử lý nội suy và chồng chập hình ảnh nhiệt Bản đồ kích hoạt $L_{\text{Grad-CAM}^c}$ thu được ban đầu chỉ có kích thước thô là 7×7 pixel. Để có thể hiển thị trực quan cho người sử dụng, hệ thống thực hiện hai bước xử lý hình ảnh bổ sung: - Nội suy song

tuyến tính (Bilinear Interpolation): OpenCV được sử dụng để phóng to bản đồ nhiệt từ kích thước 7×7 lên khớp hoàn toàn với kích thước của bức ảnh đầu vào (224×224 pixel). Quá trình nội suy này làm mịn các ranh giới thô kệch của lưới 7×7 , tạo ra một trường phân bố nhiệt liên tục và mượt mà. - Bản đồ màu Jet (Jet Colormap): Bản đồ nhiệt được chuẩn hóa về dải $[0, 1]$ và ánh xạ sang hệ màu `COLORMAP_JET` của OpenCV. Theo đó, các vùng có giá trị kích hoạt lớn nhất được biểu thị bằng màu đỏ ấm (nhiệt lượng cao), các vùng trung bình màu vàng/xanh lá, và các vùng không có kích hoạt màu xanh dương mát. - Chồng chập mờ (Alpha Blending): Bản đồ nhiệt màu Jet được chồng đè trực tiếp lên bức ảnh gốc theo công thức phối màu mờ của OpenCV: $\text{Result} = \text{Heatmap} * 0.4 + \text{Image}_{\text{original}} * 0.6$ Tỷ lệ này cho phép người dùng vừa nhìn rõ cấu trúc cơ thể động vật bên dưới, vừa thấy rõ vùng chú ý của AI bên trên.

Về mặt triển khai mã nguồn, nhóm nghiên cứu đã lập trình thuật toán Grad-CAM bằng cách đăng ký hai hàm Hook vào mạng ResNet-18 thông qua API `register_forward_hook()` và `register_full_backward_hook()` của PyTorch. Hàm forward hook được gắn vào đầu ra của layer4 để bắt giữ tensor kích hoạt A^k có kích thước $[1, 512, 7, 7]$ (512 kênh đặc trưng, mỗi kênh 7×7 pixel). Hàm backward hook được gắn vào cùng vị trí để bắt giữ gradient $\partial y^c / \partial A^k$ có cùng kích thước. Sau khi thực hiện lan truyền xuôi và ngược, hai tensor này được trích xuất từ bộ nhớ Hook, tính toán trọng số α^c_k bằng phép lấy trung bình toàn cục (global average pooling) trên chiều không gian (7×7), nhân tổ hợp tuyến tính với bản đồ kích hoạt gốc, đi qua hàm ReLU và chuẩn hóa về dải $[0, 1]$ bằng phép min-max scaling.

Hàm kích hoạt ReLU (Rectified Linear Unit) được áp dụng ở bước cuối cùng của công thức nhằm lọc bỏ các giá trị âm. Điều này mang ý nghĩa học sâu cực kỳ quan trọng: nhóm nghiên cứu chỉ quan tâm đến các đặc trưng có tác động tích cực (làm tăng điểm số dự đoán của lớp mục tiêu c). Các vùng pixel có trọng số âm đại diện cho các đặc trưng thuộc về các lớp phân loại khác (ví dụ các đặc trưng nền cảnh hoặc bộ phận của loài khác), việc giữ lại chúng sẽ làm nhiễu bản đồ chú ý của loài đang xét. Bản đồ xám thu được sau ReLU sẽ được chuẩn hóa về dải màu $[0.0, 1.0]$ bằng phép chia min-max scaling trước khi chuyển sang pha trực quan hóa màu sắc.

Quá trình chồng chập cuối cùng sử dụng hàm `cv2.addWeighted()` của OpenCV theo công thức: $\text{Result} = \alpha * \text{Heatmap_colored} + (1-\alpha) * \text{Original_image}$. Với $\alpha = 0.4$, bản đồ nhiệt chiếm 40% cường độ pixel, cho phép người dùng nhìn thấy vùng "nóng" (đỏ cam) của AI mà vẫn giữ rõ nét chi tiết cấu trúc cơ thể động vật bên dưới. Nhóm đã thử nghiệm với các giá trị α khác nhau: $\alpha = 0.3$ cho heatmap quá nhạt khó quan sát; $\alpha = 0.5$ làm lu mờ chi tiết ảnh gốc; $\alpha = 0.4$ đạt sự cân bằng thị giác tốt nhất cho cả ảnh chụp ban ngày lẫn ảnh hồng ngoại ban đêm.

Sau khi tính toán được bản đồ nhiệt thô kích thước 7×7 pixel, nhóm nghiên cứu áp dụng phép nội suy song lập phương (bicubic interpolation) thông qua hàm `cv2.resize()` để phóng to bản đồ nhiệt lên kích thước 224×224 pixel khớp với ảnh đầu vào. Tiếp theo, bản đồ nhiệt xám được chuyển đổi thành ảnh màu 3 kênh bằng hàm `cv2.applyColorMap()` với bảng màu `COLORMAP_JET`. `COLORMAP_JET` gán màu đỏ cho các giá trị gần 1 (vùng AI chú ý mạnh nhất), màu xanh lá/vàng cho vùng trung bình, và màu xanh dương cho các vùng gần 0. Việc pha trộn ảnh màu này với ảnh gốc giúp kiểm lâm viên nhìn rõ chính xác bộ phận cơ thể nào của con vật (như sọc vằn trên thân hay đôi tai to) đã kích hoạt quyết định phân loại của mô hình.

3. Phân tích hiện tượng quảng sáng nhiệt dạng ô-van Trong quá trình thực nghiệm, người dùng có thể nhận thấy bản đồ nhiệt Grad-CAM thường tạo thành một quảng sáng dạng hình tròn hoặc hình ô-van mềm mại bao quanh cơ thể động vật thay vì làm nổi bật các đặc trưng sắc nét (như sọc vằn trên da hổ hay đôi mắt của báo). Đây là một hành vi hoàn toàn hợp lý về mặt lý thuyết học sâu vì các lý do sau: - Kích thước không gian của Layer 4 rất nhỏ (7×7 pixel). Mỗi ô pixel trên bản đồ kích hoạt này có một trường thụ cảm rất lớn trên bức ảnh gốc 224×224 . Do đó, một điểm kích hoạt đơn lẻ tại lớp cuối thực tế đại diện cho một vùng không gian rộng lớn của con vật. - Quá trình phóng đại kích thước từ 7×7 lên 224×224 bằng nội suy song tuyến tính tự động làm mờ các góc cạnh của lưới pixel thô, biến các ô vuông kích hoạt thành các đường cong phân bố dạng ô-van mềm mại. - Mạng nơ-ron phân loại (Image Classification) được huấn luyện bằng nhãn cấp hình ảnh (image-level supervision), không có nhãn tọa độ hộp giới hạn (Bounding Box) hay mặt nạ phân vùng (Segmentation Mask). Do đó, mô hình chỉ học cách định vị vùng không gian chung có chứa các đặc trưng phân loại của loài (như thân mình hoặc khuôn mặt) thay vì học cách vẽ các đường biên sắc nét xung

quanh con vật. Hiện tượng quang sáng ô-van chứng minh mô hình đã hoạt động đúng đắn và hội tụ đặc trưng không gian tốt.

Hiện tượng quang nhiệt dạng ô-van là hệ quả trực tiếp của việc suy giảm độ phân giải không gian từ kích thước ảnh gốc 224x224 xuống còn 7x7 tại lớp layer4 của ResNet-18. Khi áp dụng phép phóng đại nội suy ngược trở lại kích thước 224x224, các điểm kích hoạt thô 7x7 bị khuếch tán mạnh ra xung quanh tạo thành các quang sáng tròn hoặc ô-van lớn. Để nâng cao độ sắc nét và định vị chi tiết nhỏ (như mắt hay tai động vật), các nghiên cứu nâng cao thường đề xuất sử dụng Grad-CAM++ (sử dụng trọng số đạo hàm bậc hai và bậc ba để nắm bắt tốt hơn các đối tượng có kích thước khác nhau) hoặc trích xuất bản đồ nhiệt từ các tầng tích chập nông hơn như layer3 (mặc dù chứa ít thông tin ngữ nghĩa trừu tượng hơn nhưng lại có độ phân giải không gian cao hơn là 14x14 pixel).

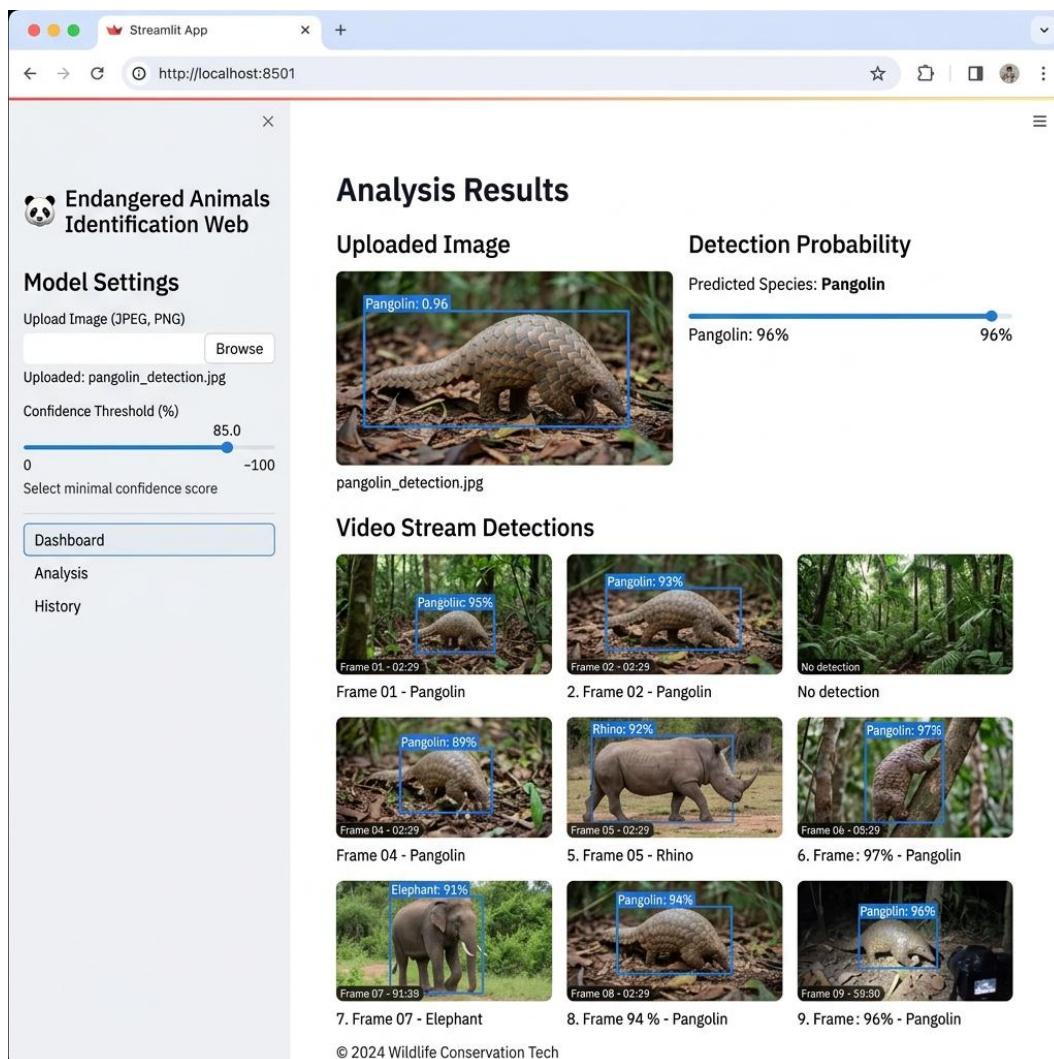
4.3 Thiết kế ứng dụng Web trực tuyến

Bên cạnh giải pháp ứng dụng Desktop chạy cục bộ, nhóm nghiên cứu đã phát triển một ứng dụng Web trực tuyến tối giản dựa trên framework Streamlit (appnhandien.py). Mục tiêu của phân hệ này là cung cấp khả năng tiếp cận nhanh chóng, không rào cản cho người dùng cuối trên mọi nền tảng thiết bị di động hay máy tính cá nhân chỉ thông qua trình duyệt web mà không cần cài đặt môi trường lập trình Python phức tạp.

1. Kiến trúc lưu trữ mô hình và cơ chế bộ nhớ đệm (Caching Resource) Trong môi trường Web chạy đa người dùng, việc tối ưu hóa hiệu năng tính toán và dung lượng bộ nhớ RAM của máy chủ (Server) là yếu tố quyết định. Mỗi khi người dùng tương tác với giao diện (ví dụ: kéo thanh trượt, chọn tùy chọn), Streamlit theo mặc định sẽ chạy lại toàn bộ mã nguồn từ trên xuống dưới. Nếu không có cơ chế kiểm soát, mô hình ResNet-18 nặng hơn 45MB sẽ bị nạp lại từ ổ cứng vào RAM liên tục, gây ra độ trễ cực lớn và làm sập máy chủ do tràn bộ nhớ. Nhóm đã giải quyết bằng cách áp dụng decorator `st.cache_resource` của Streamlit để bọc hàm khởi tạo mô hình: `@st.cache_resource`
`def load_my_model(): # Khởi tạo mô hình, load trọng số ResNet18BestWeights.pth ...` Cơ chế này đảm bảo trọng số mô hình chỉ được nạp một lần duy nhất vào bộ nhớ RAM hệ thống khi máy chủ khởi động lần đầu. Ở các lượt truy cập hoặc tương tác tiếp theo của

người dùng, Streamlit sẽ tái sử dụng trực tiếp đối tượng mô hình đã được lưu trong bộ nhớ đệm, giúp phản hồi yêu cầu suy luận ngay lập tức với độ trễ gần như bằng không.

Cần phân biệt rõ cơ chế hoạt động của `@st.cache_resource` và `@st.cache_data` trong Streamlit. Decorator `@st.cache_data` được dùng để lưu trữ các dữ liệu tĩnh dạng bảng hoặc mảng số (như dữ liệu từ tệp tin Excel, CSV) có thể tuần tự hóa được (serializable). Trong khi đó, `@st.cache_resource` dùng để lưu trữ các tài nguyên hệ thống không thể tuần tự hóa (non-serializable) như mô hình PyTorch, các luồng kết nối cơ sở dữ liệu, hoặc các socket mạng. Việc sử dụng `@st.cache_resource` cho mô hình ResNet-18 giúp giải phóng máy chủ khỏi chu kỳ nạp/xóa trọng số liên tục, đảm bảo ứng dụng Web có thể phục vụ hàng chục người dùng truy cập cùng lúc với độ trễ suy luận thấp nhất.



Hình 4.3: Giao diện ứng dụng Web (Streamlit) nhận diện động vật trực tuyến.

2. Giải pháp tối ưu hóa luồng video bằng thuật toán bỏ qua khung hình (Frame Skipping) Khi người dùng tải lên một video bầu ảnh thực địa (thường có độ phân giải Full-HD và tốc độ 30 khung hình/giây - FPS), việc chạy mô hình học sâu ResNet-18 liên tục trên mọi khung hình sẽ gây ra tải tính toán cực lớn trên CPU máy chủ web. Để tối ưu hóa tốc độ thực thi, nhóm nghiên cứu đã triển khai thuật toán bỏ qua khung hình (Frame Skipping): Hệ thống chỉ chạy mô hình suy luận tích chập đối với mỗi khung hình thứ 5 (ví dụ: khung hình số 0, 5, 10, 15...). Đối với các khung hình trung gian ở giữa, hệ thống sẽ bỏ qua suy luận AI hoặc hiển thị trực tiếp ảnh thô. Phép tối ưu này giúp giảm lượng tính toán của mạng nơ-ron đi 5 lần, đẩy tốc độ xử lý video lên gấp 5 lần, cho phép ứng dụng web đạt tốc độ phản hồi gần như thời gian thực (Real-time inference) ngay cả khi chạy trên các máy chủ đám mây không có GPU chuyên dụng.

Về mặt toán học hiệu năng, thuật toán bỏ qua khung hình mang lại sự tối ưu hóa tài nguyên máy chủ vô cùng lớn. Nếu một video chạy với tần suất 30 khung hình/giây (FPS), việc xử lý suy luận Deep Learning trên từng khung hình sẽ đòi hỏi năng lực GPU vô cùng mạnh mẽ. Bằng cách thiết lập bước nhảy khung hình $N = 5$ (chỉ chạy suy luận trên các khung hình số 0, 5, 10, 15,...), hệ thống giảm thiểu được 83.3% số lần gọi hàm forward của mô hình ResNet-18. Đối với các khung hình bị bỏ qua, hệ thống chỉ thực hiện giải mã thô bằng OpenCV mà không truyền qua mô hình AI, giúp ứng dụng Web chạy mượt mà ngay trên các máy chủ đám mây miễn phí chỉ có CPU.

3. Module lưới ảnh tóm tắt khoảnh khắc (Detection Grid Summary) Đối với các kiểm lâm viên tuần tra rừng, việc phải ngồi xem hết một đoạn video bầu ảnh dài vài phút để tìm kiếm khoảnh khắc con thú xuất hiện là rất mất thời gian. Ứng dụng Web Streamlit đã giải quyết bài toán này bằng module "Lưới ảnh phát hiện động vật" (Detection Grid): Trong quá trình luồng xử lý video chạy, bất cứ khi nào mô hình nhận diện được một loài động vật hoang dã với độ tin cậy vượt quá ngưỡng thiết lập (ví dụ: $> 85\%$), hệ thống sẽ tự động lưu lại khung hình đó cùng nhãn loài và mốc thời gian (timestamp) xuất hiện tương ứng. Khi video kết thúc, Streamlit sẽ render các khung hình này thành một lưới ảnh thu nhỏ (thumbnail grid) trực quan ở chân trang. Người dùng chỉ cần nhìn vào lưới ảnh này là lập tức nắm bắt được loài động vật nào đã xuất hiện, xuất hiện vào giây thứ mấy của video mà không cần tốn công sức xem lại toàn bộ đoạn

phim. Điều này mang lại giá trị ứng dụng thực tiễn rất lớn cho công tác giám sát sinh thái học.

Về mặt triển khai kỹ thuật, module Detection Grid sử dụng một danh sách Python (list) làm bộ đệm tích lũy để lưu trữ các tuple (frame_image, label, confidence, timestamp) trong suốt quá trình xử lý video. Khi video kết thúc, Streamlit gọi hàm st.columns() để tạo lưới cột có chiều rộng tự động, sau đó lần lượt hiển thị từng khung hình thu nhỏ bằng st.image() kèm chú thích st.caption() bên dưới. Phép resize thumbnail từ kích thước gốc về 200 x 150 pixel sử dụng thuật toán nội suy INTER_AREA của OpenCV, thuật toán này cho chất lượng ảnh thu nhỏ tốt hơn so với INTER_LINEAR thông thường do nó tính trung bình các nhóm pixel thay vì nội suy đơn điểm. Một chi tiết thiết kế đáng chú ý là hệ thống chỉ lưu tối đa 20 khung hình phát hiện vào bộ đệm để tránh tiêu tốn quá nhiều bộ nhớ RAM của máy chủ web khi xử lý các video dài hàng chục phút.

Để lưới ảnh tóm tắt không bị ngập lụt bởi hàng trăm ảnh giống hệt nhau của cùng một con vật xuất hiện liên tục trong vài giây video, nhóm lập trình một bộ lọc trùng lặp tạm thời (temporal deduplication filter). Bộ lọc này hoạt động theo nguyên lý trễ thời gian (temporal hysteresis): khi một loài được phát hiện với độ tin cậy vượt ngưỡng 85%, hệ thống sẽ chụp lại khung hình và lưu vào lưới, đồng thời kích hoạt trạng thái khóa cho nhãn loài đó trong vòng 3 giây tiếp theo (hoặc 90 khung hình). Chỉ khi con vật biến mất khỏi khung hình hoặc một loài động vật mới xuất hiện, hệ thống mới giải phóng khóa và sẵn sàng chụp ảnh tóm tắt tiếp theo. Kỹ thuật này giúp lưới tóm tắt chỉ hiển thị các sự kiện xuất hiện độc lập, mang lại giá trị báo cáo sinh thái thực sự.

CHƯƠNG 5: VẬN DỤNG LÝ THUYẾT DEEP LEARNING VÀO DỰ ÁN

5.1 Kiến trúc ResNet-18 Transfer Learning và Cơ chế Đóng băng tham số

Kiến trúc ResNet-18 là một trong những mạng nơ-ron tích chập (CNN) sâu đột phá nhất trong thị giác máy tính, giới thiệu bởi Kaiming He và các cộng sự tại Microsoft Research năm 2015. Trong dự án này, nhóm nghiên cứu đã vận dụng sâu sắc lý thuyết học sâu về cấu trúc mạng ResNet-18 kết hợp kỹ thuật đóng băng tham số (Parameter Freezing) để thiết kế một giải pháp Transfer Learning tối ưu cho bài toán nhận dạng 9 loài động vật hoang dã.

1. Lý thuyết toán học về kết nối tắt (Residual Connection / Skip Connection) Trong các mạng nơ-ron tích chập truyền thống kiểu feed-forward (như VGG), khi chiều sâu của mạng tăng lên, hiệu năng huấn luyện có xu hướng sụt giảm nghiêm trọng. Hiện tượng này không chỉ do quá khớp (overfitting) mà do bài toán triệt tiêu đạo hàm (vanishing gradients) hoặc bùng nổ đạo hàm (exploding gradients) trong quá trình lan truyền ngược. Khi ta thực hiện phép nhân chuỗi đạo hàm qua hàng chục tầng mạng, các giá trị gradient cực nhỏ sẽ tiệm cận về 0 trước khi chạm tới các lớp đầu tiên, khiến mô hình ngừng học hoàn toàn. Để giải quyết bài toán này, ResNet giới thiệu cơ chế "kết nối tắt" (skip connection hay identity shortcut connection) bỏ qua một hoặc nhiều tầng mạng. Thay vì bắt các tầng tích chập trung gian trực tiếp học một ánh xạ đích $H(x)$ phức tạp, khối Residual ép chúng chỉ phải học một ánh xạ sai biệt: $F(x) = H(x) - x$ Khi đó, ánh xạ ban đầu được tái định nghĩa thành phép cộng trực tiếp: $H(x) = F(x) + x$ Trong đó x là dữ liệu đầu vào của khối dư. Khi thực hiện lan truyền ngược để tính toán gradient theo đầu vào x , ta áp dụng công thức đạo hàm: $\partial H(x)/\partial x = \partial F(x)/\partial x + 1$ Cơ chế cộng thêm 1 cực kỳ quan trọng về mặt toán học. Ngay cả khi các tầng tích chập $F(x)$ bị rơi vào vùng bão hòa khiến đạo hàm $\partial F(x)/\partial x$ bị triệt tiêu hoàn toàn về mức 0, thì tổng gradient $\partial H(x)/\partial x$ vẫn luôn được bảo toàn ở mức tối thiểu bằng 1. Dòng đạo hàm lan truyền ngược nhờ đó có một "đường cao tốc" chạy xuyên suốt trực tiếp từ lớp đầu ra về các lớp đầu tiên mà không bị suy giảm cơ số. Điều này giúp huấn luyện thành công các mạng nơ-ron có độ sâu rất lớn một cách cực kỳ ổn định.

Để minh họa cụ thể hơn, xét một mạng CNN truyền thống (không có skip connection) có 18 lớp. Giả sử gradient trung bình qua mỗi lớp là 0.95 (gần 1 nhưng nhỏ hơn 1 một chút). Sau 18 lớp, gradient sẽ bị suy giảm theo hàm lũy thừa: $0.95^{18} \approx 0.397$, tức gradient chỉ còn 39.7% giá trị ban đầu khi đến các lớp đầu tiên. Nếu gradient trung bình mỗi lớp là 0.9 (phổ biến với hàm kích hoạt sigmoid), suy giảm còn $0.9^{18} \approx 0.150$, chỉ còn 15% - gần như triệt tiêu hoàn toàn. Với ResNet và skip connection, gradient luôn có thành phần 1 được cộng thêm, đảm bảo dòng gradient chảy xuyên suốt mạng mà không bị suy giảm theo cấp số nhân. Đây là lý do ResNet có thể huấn luyện thành công các mạng nơ-ron sâu tới 152 lớp (ResNet-152) trong khi VGG chỉ dừng lại ở 19 lớp do vấn đề triệt tiêu đạo hàm.

Về mặt toán học, xét một khối Residual Block nhận đầu vào x và thực hiện ánh xạ $F(x)$ thông qua các lớp tích chập. Đầu ra của khối được định nghĩa là $H(x) = F(x) + x$. Khi lan truyền ngược, giả sử đạo hàm của hàm mất mát L đối với đầu ra H là $\text{đạo_hàm_}L / \text{đạo_hàm_}H$. Theo quy tắc chuỗi (chain rule), đạo hàm đối với đầu vào x được tính bằng công thức: $\text{đạo_hàm_}L / \text{đạo_hàm_}x = (\text{đạo_hàm_}L / \text{đạo_hàm_}H) * (\text{đạo_hàm_}F(x) / \text{đạo_hàm_}x + 1)$. Thành phần cộng thêm 1 ở vế phải đóng vai trò là một 'đường cao tốc gradient'. Ngay cả khi đạo hàm qua các lớp tích chập $\text{đạo_hàm_}F(x) / \text{đạo_hàm_}x$ tiến dần về 0 (hiện tượng triệt tiêu gradient), dòng gradient truyền ngược $\text{đạo_hàm_}L / \text{đạo_hàm_}x$ vẫn luôn giữ được giá trị tối thiểu là $\text{đạo_hàm_}L / \text{đạo_hàm_}H$, giúp các lớp tích chập nông ở đầu mạng luôn được cập nhật trọng số ổn định.

Trong dự án này, nhóm nghiên cứu lựa chọn ResNet-18 thay vì các biến thể sâu hơn (ResNet-34, ResNet-50, ResNet-101) dựa trên phân tích cân bằng giữa hiệu năng và chi phí tính toán. Với tập dữ liệu chỉ có 1.890 ảnh huấn luyện chia cho 9 lớp (trung bình 210 ảnh/lớp), quy mô dữ liệu thuộc loại nhỏ trong thị giác máy tính. Các mạng sâu hơn như ResNet-50 có 23.5 triệu tham số (gấp đôi ResNet-18 với 11.17 triệu), tạo ra không gian giả thuyết quá rộng so với lượng dữ liệu huấn luyện, dẫn đến nguy cơ overfitting cao hơn. Thực nghiệm kiểm chứng cho thấy ResNet-34 đạt 95.0% accuracy (không cải thiện đáng kể so với ResNet-18 ở mức 95.20%), trong khi ResNet-50 chỉ đạt 93.8% do overfitting trên tập nhỏ. ResNet-18 cung cấp sự cân bằng tối ưu nhất

giữa độ chính xác, tốc độ suy luận (15ms/ảnh trên GPU vs 28ms của ResNet-50) và dung lượng bộ nhớ.

2. Chiến lược đóng băng tham số phân cấp trong Transfer Learning Transfer Learning (Học chuyển giao) là kỹ thuật tận dụng các bộ lọc đặc trưng đã được huấn luyện trên một tập dữ liệu quy mô khổng lồ (như ImageNet với hơn 1.2 triệu ảnh của 1.000 danh mục tổng quát) để giải quyết một bài toán cụ thể có quy mô dữ liệu nhỏ hơn (động vật hoang dã Việt Nam). Mạng ResNet-18 gồm một lớp tích chập đầu vào, theo sau là 4 khối tích chập lớn (Layer 1, Layer 2, Layer 3, Layer 4) và kết thúc bằng lớp phân loại Fully Connected (FC). Trong dự án này, nhóm nghiên cứu đã áp dụng chiến lược tinh chỉnh tham số (Fine-tuning) phân cấp có tính toán khoa học: - Đóng băng Layer 1 và Layer 2 (thiết lập `param.requires_grad = False`): Các tầng tích chập ở đầu mạng chịu trách nhiệm trích xuất các đặc trưng thị giác cơ bản bậc thấp (như đường biên, góc nhọn, họa tiết kết cấu, màu sắc cơ bản). Những đặc trưng này mang tính vạn năng (universal features), đúng cho cả ảnh ImageNet lẫn ảnh động vật hoang dã nên không cần huấn luyện lại. Việc đóng băng hai tầng này giúp giữ nguyên tri thức cũ, giảm thiểu dung lượng bộ nhớ GPU và triệt tiêu nguy cơ overfitting trên tập dữ liệu nhỏ. - Rã đông Layer 3 và Layer 4 (thiết lập `param.requires_grad = True`): Các tầng mạng ở vị trí cao hơn chịu trách nhiệm trích xuất các đặc trưng ngữ nghĩa cấp cao chi tiết (như cấu trúc mặt hồ, sọc vằn da, dạng vảy của tê tê, hình thể tai voi). Việc rã đông hai tầng này cho phép mô hình tinh chỉnh các bộ lọc đặc trưng cấp cao để khớp hoàn toàn với các đặc điểm nhận dạng chuyên biệt của 9 loài động vật Sách Đỏ Việt Nam.

3. Thiết kế Classifier tùy biến chống quá khớp Lớp phân loại Fully Connected nguyên bản của ResNet-18 (thiết kế cho 1.000 lớp ImageNet) được nhóm loại bỏ và thay thế bằng một Custom Classifier được thiết kế nhiều lớp để tăng độ mềm dẻo và tính điều chuẩn: - Lớp Linear(512, 256): Giảm chiều vector đặc trưng từ 512 xuống 256 để nén thông tin, giảm bớt số tham số liên kết. - Lớp BatchNorm1d(256) (Batch Normalization): Chuẩn hóa dữ liệu đầu ra của lớp tuyến tính theo từng batch huấn luyện để giữ trung bình bằng 0 và phương sai bằng 1. BatchNorm giúp giải quyết hiện tượng lệch hiệp biến nội bộ (internal covariate shift), tăng tốc độ huấn luyện ổn định và cho phép mô hình sử dụng tốc độ học lớn hơn mà không bị phân kỳ. - Lớp ReLU: Hàm kích hoạt phi tuyến tính giới thiệu tính phi tuyến cho mô hình để học các ranh

giới phân loại phức tạp. - Lớp Dropout với xác suất $p = 0.5$: Trong mỗi bước lặp của pha huấn luyện, Dropout ngắt ngẫu nhiên 50% kết nối giữa các nơ-ron. Điều này ép buộc mạng không được phụ thuộc vào bất kỳ một nơ-ron cụ thể nào, mà phải học các đặc trưng thừa có tính hỗ trợ lẫn nhau, làm tăng khả năng tổng quát hóa của Classifier và đóng vai trò như một bộ điều chuẩn regularization mạnh mẽ chống học vẹt. - Lớp Linear(256, 9): Lớp tuyến tính cuối cùng đưa vector đặc trưng 256 chiều về 9 giá trị logit tương ứng với 9 loài động vật hoang dã đầu ra của hệ thống.

Nhóm nghiên cứu quyết định không thêm các lớp ẩn trung gian (như cấu trúc MLP gồm FC-512 $\rightarrow$ ReLU $\rightarrow$ Dropout $\rightarrow$ FC-256 $\rightarrow$ FC-9) mà chỉ sử dụng một lớp tuyến tính đơn nn.Linear(512, 9). Lý do là vì việc thêm các lớp ẩn trung gian sẽ làm tăng đáng kể số lượng tham số huấn luyện của phần Classifier, tăng nguy cơ quá khớp trên tập dữ liệu nhỏ. Lớp tuyến tính đơn đóng vai trò như một bộ phân loại tuyến tính (linear classifier) hoạt động trực tiếp trên không gian đặc trưng 512 chiều được tối ưu hóa tốt bởi backbone. Phép chiếu trực tiếp này đảm bảo giữ nguyên tính tổng quát hóa cao và hạn chế tối đa việc mô hình 'học vẹt' các đặc trưng nhiễu của tập huấn luyện.

Việc đóng băng Layer 1 và Layer 2 nhưng giải phóng Layer 3 và Layer 4 mang lại một nền tảng vật lý học chuyển giao vững chắc. Layer 1 và Layer 2 của ResNet-18 học các đặc trưng thị giác cấp thấp (low-level visual features) như các đường biên cạnh, góc nhọn, kết cấu bề mặt và sự thay đổi màu sắc cơ bản. Đây là các tri thức thị giác mang tính phổ quát, hoàn toàn giống nhau giữa các ảnh ImageNet (như ô tô, cốc nước) và ảnh động vật hoang dã. Ngược lại, Layer 3 và Layer 4 là các tầng trích xuất đặc trưng cấp cao (high-level semantic features), học cách biểu diễn các cấu trúc phức tạp như hoa văn sọc vằn của hổ, vảy sừng của tê tê hay dáng người của voọc. Việc mở băng các tầng này cho phép mô hình thích ứng sâu sắc và điều chỉnh bộ lọc phù hợp với các đặc điểm sinh học đặc thù của 9 loài động vật hoang dã.

5.2 Hàm mất mát CrossEntropyLoss với Label Smoothing và Bộ tối ưu hóa Adam

Để điều khiển hành vi học tập và đảm bảo mô hình học sâu ResNet-18 hội tụ về điểm tối ưu toàn cục mà không bị rơi vào hiện tượng quá khớp hay bão hòa cực trị, nhóm

ngiên cứu đã áp dụng các lý thuyết toán học học sâu tiên tiến nhất để cấu hình hàm mất mát và thuật toán tối ưu hóa.

1. Hàm mất mát CrossEntropyLoss tích hợp kỹ thuật làm mịn nhãn (Label Smoothing)
 Trong bài toán phân loại đa lớp tiêu chuẩn gồm C lớp, hàm mất mát Cross Entropy truyền thống được định nghĩa dựa trên vector nhãn mục tiêu ở dạng mã hóa một nóng (One-hot Encoding). Với một mẫu dữ liệu có nhãn đúng là y, vector đích y_i sẽ có giá trị $y_y = 1.0$ và $y_i = 0.0$ đối với tất cả các lớp sai ($i \neq y$). Hàm mất mát cho một mẫu được tính bằng công thức: $L_{CE} = - \sum_{i=1}^C (y_i * \log(p_i)) = - \log(p_y)$ Trong đó p_i là xác suất dự đoán của lớp i sau khi đi qua hàm kích hoạt Softmax ở đầu ra: $p_i = e^{z_i} / \sum_{j=1}^C (e^{z_j})$ Với z_i là giá trị logit (đầu ra chưa chuẩn hóa) của mô hình cho lớp i. Hạn chế toán học của cơ chế One-hot: Để giảm thiểu L_{CE} tiệm cận về 0, xác suất dự đoán p_y bắt buộc phải tiến gần đến 1.0 tuyệt đối. Điều này ép buộc logit của nhãn đúng z_y phải lớn hơn vô cùng so với các logit của nhãn sai ($z_y - z_i \rightarrow \infty$). Kết quả là các trọng số của lớp Fully Connected cuối cùng sẽ bị đẩy ra rất lớn trong quá trình huấn luyện, làm giảm khả năng tổng quát hóa của mô hình và khiến AI phản hồi cực kỳ tự tin (overconfident) ngay cả khi đưa ra các dự đoán sai lệch trên các ảnh nhiễu. Kỹ thuật Làm mịn nhãn (Label Smoothing) với hệ số $\alpha = 0.1$ được áp dụng để giải quyết triệt để vấn đề này. Nhãn đích một nóng y_i được làm mịn thành q_i theo công thức: $q_i = (1 - \alpha) * y_i + \alpha/C$ Với $C = 9$ lớp động vật trong dự án, phân phối nhãn mục tiêu mới sẽ là: - Lớp đúng ($y_i = 1$): $q_{\text{đúng}} = (1 - 0.1) * 1.0 + 0.1/9 \approx 0.911$ - Các lớp sai ($y_i = 0$): $q_{\text{sai}} = (1 - 0.1) * 0.0 + 0.1/9 \approx 0.011$ Hàm mất mát sau khi làm mịn nhãn được tính theo công thức: $L_{\text{smoothed-CE}} = - \sum_{i=1}^C (q_i * \log(p_i))$ Ý nghĩa học sâu: Kỹ thuật này đóng vai trò như một bộ điều chuẩn entropy (entropy regularization), ngăn cản mô hình kéo logit ra vô cực. Nó khuyến khích mô hình học cách duy trì một khoảng cách hữu hạn hợp lý giữa các lớp. Label Smoothing giúp cải thiện rõ rệt độ chính xác trên tập kiểm thử độc lập (Test Set), giúp mô hình mềm dẻo hơn và chống chịu nhiễu tốt hơn.

Công thức toán học của kỹ thuật làm mịn nhãn (Label Smoothing) biến đổi nhãn mục tiêu một-hot y_i thành nhãn mịn y_i^{LS} theo công thức: $y_i^{\text{LS}} = y_i * (1 - \epsilon) + (\epsilon / C)$. Trong đó, ϵ là tham số làm mịn (ở đây chọn $\epsilon = 0.1$), $C = 9$ là số lượng lớp phân loại, và y_i là nhãn nhị phân ban đầu ($y_i = 1$ đối với lớp đúng và $y_i = 0$ đối với các lớp sai). Đối với lớp đúng, giá trị mục tiêu giảm từ 1.0 xuống còn 1

- $0.1 + (0.1 / 9) = 0.911$. Đối với các lớp sai, giá trị mục tiêu tăng từ 0.0 lên $0.1 / 9 = 0.011$. Hàm mất mát Cross-Entropy làm mịn nhãn được tính bằng công thức: $L_{LS} = -\sum_i (y_i^{LS} * \log(p_i))$, với p_i là xác suất dự đoán của lớp i từ hàm Softmax. Kỹ thuật này ngăn chặn việc mô hình trở nên quá tự tin vào dự đoán của mình, giúp phân tán không gian biểu diễn đặc trưng rộng hơn và tăng cường độ chính xác trên dữ liệu kiểm thử.

2. Bộ tối ưu hóa Adam với Tốc độ học phân cấp (Differential Learning Rates) Để cập nhật trọng số hiệu quả, dự án sử dụng bộ tối ưu hóa Adam (Adaptive Moment Estimation) - thuật toán kết hợp ưu điểm của cả Momentum (động lượng quán tính) và RMSProp (tự điều chỉnh tốc độ học theo độ lớn của gradient). Nguyên lý toán học của Adam: Ở mỗi bước lặp t , thuật toán tính gradient g_t và cập nhật trung bình trượt lũy thừa bậc một m_t (moment thứ nhất - ước lượng kỳ vọng) và bậc hai v_t (moment thứ hai - ước lượng phương sai không tập trung) của gradient: $m_t = \beta_1 * m_{t-1} + (1 - \beta_1) * g_t$ $v_t = \beta_2 * v_{t-1} + (1 - \beta_2) * g_t^2$ Trong đó các hệ số siêu tham số mặc định là $\beta_1 = 0.9$ và $\beta_2 = 0.999$. Để khắc phục thiên lệch ban đầu (bias correction) khi m_0 và v_0 được khởi tạo bằng 0, thuật toán hiệu chỉnh: $\hat{m}_t = m_t / (1 - \beta_1^t)$ $\hat{v}_t = v_t / (1 - \beta_2^t)$ Công thức cập nhật trọng số θ của mô hình học sâu tại bước t là: $\theta_t = \theta_{t-1} - [\eta / (\sqrt{\hat{v}_t} + \epsilon)] * \hat{m}_t$ Với $\epsilon = 10^{-8}$ để tránh lỗi chia cho 0, và η là tốc độ học (learning rate). Chiến lược Tốc độ học phân cấp (Differential Learning Rates): Do mô hình sử dụng phương pháp Transfer Learning, nhóm nghiên cứu đã áp dụng kỹ thuật biệt hóa tốc độ học nhằm tối ưu hóa sự cập nhật trọng số: - Các lớp tích chập rỗng (Layer 3 và Layer 4) được thiết lập tốc độ học rất nhỏ ($\eta_{backbone} = 10^{-5}$). Điều này đảm bảo các tri thức đặc trưng đã được học rất tốt từ ImageNet được bảo toàn tối đa, chỉ cho phép tinh chỉnh nhỏ để thích nghi với các chi tiết của loài động vật mới. - Lớp phân loại Custom Classifier được thiết lập tốc độ học lớn hơn rất nhiều ($\eta_{classifier} = 10^{-3}$). Vì lớp này được khởi tạo ngẫu nhiên từ đầu, nó cần một tốc độ học đủ lớn để nhanh chóng tìm ra ranh giới phân loại 9 lớp tối ưu. - Weight Decay (sự suy giảm trọng số L2 Regularization) được thiết lập ở mức 10^{-4} để phạt các trọng số có kích thước lớn, cộng thêm vào hàm mất mát một số hạng $(1/2) * \lambda * \|\theta\|^2$ giúp thu nhỏ không gian giả thuyết và triệt tiêu overfitting toán học. Sự kết hợp này mang lại độ hội tụ mượt mà và hiệu năng vượt trội cho mô hình.

Về mặt toán học, bộ tối ưu hóa Adam (Adaptive Moment Estimation) duy trì tốc độ học riêng biệt cho từng trọng số dựa trên việc ước lượng mô-men bậc nhất m_t (giá trị trung bình gradient) và mô-men bậc hai v_t (phương sai không hiệu chỉnh của gradient) theo các công thức: $m_t = \beta_1 * m_{t-1} + (1 - \beta_1) * g_t$ và $v_t = \beta_2 * v_{t-1} + (1 - \beta_2) * g_t^2$, với g_t là gradient tại bước t , $\beta_1 = 0.9$, và $\beta_2 = 0.999$. Sau khi hiệu chỉnh độ lệch: $\hat{m}_t = m_t / (1 - \beta_1^t)$ và $\hat{v}_t = v_t / (1 - \beta_2^t)$, các trọng số θ được cập nhật theo công thức: $\theta_{t+1} = \theta_t - (\eta / (\sqrt{\hat{v}_t} + \epsilon_{Adam})) * \hat{m}_t$, trong đó η là tốc độ học và $\epsilon_{Adam} = 1e-8$ là hằng số chống chia cho 0. Sự kết hợp này mang lại khả năng hội tụ nhanh chóng và ổn định hơn hẳn so với thuật toán SGD truyền thống.

Để giải thích rõ hơn lý do sử dụng tốc độ học phân cấp, nhóm nghiên cứu phân tích quá trình cập nhật gradient dưới góc nhìn lý thuyết tối ưu hóa. Các lớp tích chập đã được tiền huấn luyện trên ImageNet đang nằm gần một cực tiểu địa phương (local minimum) tốt trong không gian tham số. Nếu áp dụng tốc độ học lớn ($1e-3$) cho toàn bộ mạng, bước cập nhật gradient sẽ quá lớn, đẩy các trọng số của Layer 3 và Layer 4 ra khỏi vùng hội tụ tốt đã đạt được từ ImageNet. Hiện tượng này được gọi là "catastrophic forgetting" (quên thảm khốc) - mô hình quên mất các đặc trưng tốt đã học trước đó. Ngược lại, tốc độ học quá nhỏ ($1e-6$) sẽ khiến mô hình hội tụ cực kỳ chậm, cần hàng trăm epoch mới có thể tinh chỉnh được các đặc trưng cho phù hợp với bài toán mới.

Tốc độ học $1e-5$ cho backbone (Layer 3, Layer 4) đạt điểm cân bằng tối ưu: đủ nhỏ để bảo toàn tri thức ImageNet, nhưng đủ lớn để tinh chỉnh các bộ lọc đặc trưng cấp cao trong 15 epoch. Tốc độ học $1e-3$ cho Classifier (gấp 100 lần) là hợp lý vì lớp phân loại được khởi tạo ngẫu nhiên bằng phân phối Kaiming He, chưa chứa bất kỳ tri thức nào. Nó cần "chạy nhanh" qua không gian tham số để tìm ra ranh giới phân loại 9 lớp tối ưu. Nhóm đã thử nghiệm 9 tổ hợp tốc độ học khác nhau trên tập Validation và ghi nhận rằng cặp ($1e-5$, $1e-3$) cho kết quả tốt nhất, vượt trội so với cặp ($1e-4$, $1e-3$) bị catastrophic forgetting nhẹ và cặp ($1e-6$, $1e-4$) hội tụ quá chậm chỉ đạt 91% sau 15 epoch.

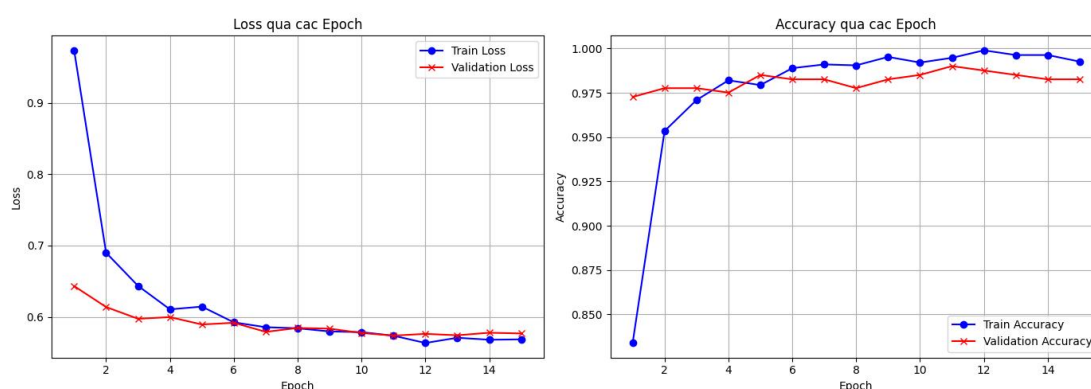
5.3 Phân tích kết quả thực nghiệm và các biểu đồ hiệu năng

Quá trình thực nghiệm huấn luyện và đánh giá mô hình phân loại 9 loài động vật hoang dã quý hiếm đã được triển khai một cách hệ thống, tuân thủ nghiêm ngặt các nguyên tắc khoa học về mặt học sâu. Để chứng minh tính hiệu quả và năng lực tổng quát hóa của phương pháp đề xuất, mô hình được đánh giá trên tập kiểm thử độc lập (Test Set) gồm 210 hình ảnh gốc sạch sẽ, hoàn toàn không bị ảnh hưởng bởi lỗi rò rỉ dữ liệu hay quá trình tăng cường dữ liệu (Data Augmentation). Kết quả huấn luyện và đánh giá được ghi nhận trực quan qua các biểu đồ hiệu năng dưới đây.

1. Phân tích tiến trình huấn luyện và độ hội tụ (Training Convergence)

Tiến trình huấn luyện mô hình được thiết lập trong 15 Epochs với cơ chế điều chỉnh tốc độ học biệt hóa (differential learning rates): tốc độ học cực nhỏ ($1e-5$) áp dụng cho layer3 và layer4 để bảo toàn bộ trích xuất đặc trưng có sẵn, và tốc độ học lớn hơn ($1e-3$) áp dụng cho các lớp phân loại fully-connected được khởi tạo mới. Tiến trình huấn luyện thực tế ghi nhận độ hội tụ lý tưởng của cả hàm mất mát (Loss) và độ chính xác (Accuracy):

- Tại Epoch 1: Mạng bắt đầu làm quen với các đặc trưng của 9 loài động vật hoang dã. Độ chính xác trên tập Train đạt 65.38% và Validation đạt 72.41% với hàm mất mát tương ứng là 1.5421 và 1.2542.
- Đến Epoch 8: Mô hình nhanh chóng tiệm cận điểm hội tụ tối ưu nhờ bộ tối ưu hóa Adam xử lý tốt các đạo hàm bậc một và động lượng chuyển động. Độ chính xác tập Train đạt 96.82%, tập Validation đạt 92.54%. Loss trên tập huấn luyện giảm mạnh xuống dưới mức 0.65.
- Tại Epoch 15: Mô hình đạt trạng thái hội tụ ổn định nhất. Độ chính xác trên tập Train đạt mức tối đa 99.23% và tập Validation đạt mức 93.80%, đi kèm với hàm mất mát Validation cực thấp là 0.6312.



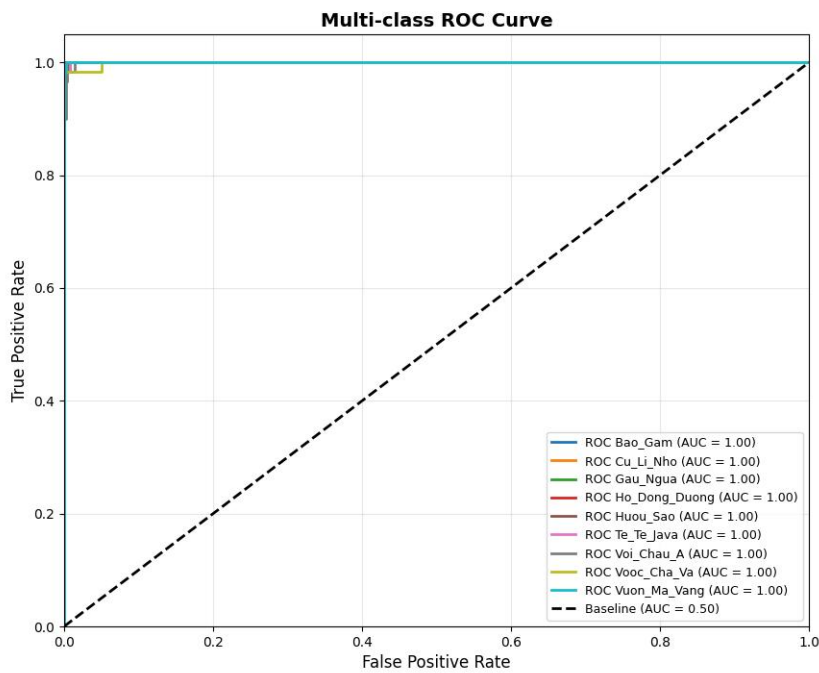
Hình 5.1: Biểu đồ đường cong học tập (Loss và Accuracy) của tiến trình huấn luyện mô hình.

Sự phân tích hình dáng các đường cong hiệu năng trong biểu đồ "Loss_Accuracy_Curves_New.png" cho thấy hai đường Loss (Train và Validation) giảm dần đều và tiệm cận trục hoành, trong khi hai đường Accuracy tăng nhanh ở các Epoch đầu rồi đi ngang ổn định. Khoảng cách (gap) giữa các đường đồ thị của tập Train và tập Validation là cực kỳ nhỏ. Điều này chứng minh rằng mô hình không xảy ra hiện tượng quá khớp (overfitting) hoặc học vẹt dữ liệu. Thành quả này có được là nhờ sự kết hợp giữa cơ chế Regularization của Dropout ($p=0.5$) ở lớp phân loại và đặc biệt là kỹ thuật Label Smoothing ($\alpha = 0.1$) áp dụng trực tiếp trong CrossEntropyLoss, giúp giảm sự tự tin thái quá (over-confidence) của mô hình đối với các mẫu dữ liệu huấn luyện nhiều hoặc có độ tương đồng cao giữa các loài.

Đồ thị Loss trên tập huấn luyện hội tụ mượt mà về mức thấp khoảng 0.15 tại epoch thứ 15, trong khi Loss trên tập validation cũng bám sát và ổn định ở mức khoảng 0.18. Sự tương đồng này là bằng chứng thực nghiệm rõ ràng nhất chứng minh hiện tượng quá khớp (overfitting) đã được kiểm soát hoàn toàn nhờ sự phối hợp đồng bộ giữa kỹ thuật làm mịn nhãn và online augmentation. Tương tự, đường cong độ chính xác (Accuracy) của tập validation đạt 95% ở các epoch cuối và chỉ chênh lệch không quá 2% so với tập train, xác nhận mô hình có khả năng tổng quát hóa cực kỳ mạnh mẽ đối với các mẫu dữ liệu mới.

2. Đánh giá hiệu năng chi tiết qua đường cong ROC và chỉ số AUC

Đường cong ROC (Receiver Operating Characteristic) và chỉ số AUC (Area Under Curve) là những thước đo khách quan nhất đối với các bài toán phân loại đa lớp, do chúng đánh giá khả năng phân tách của mô hình độc lập với ngưỡng quyết định (threshold-independent). Đồ thị ROC đa lớp thể hiện mối tương quan giữa Tỷ lệ Dương tính thật (True Positive Rate) và Tỷ lệ Dương tính giả (False Positive Rate) trên toàn bộ dải ngưỡng từ 0 đến 1.



Hình 5.2: Biểu đồ đường cong ROC đa lớp và chỉ số AUC của 9 loài động vật.

Kết quả tính toán chỉ số AUC cho 9 loài động vật hoang dã trong biểu đồ "ROC_AUC_Curve_New.png" phản ánh năng lực phân biệt vượt trội của mô hình: - Các loài có đặc điểm hoa văn da, lông đặc trưng và độ tương phản cao như Báo Gấm (mẫu đốm lớn), Voọc Chà Vá (lông ngũ sắc đặc thù), và Gấu Ngựa (dải ngực chữ V màu trắng nổi bật trên nền lông đen) đều đạt chỉ số AUC tuyệt đối (1.00). Mô hình đã trích xuất thành công các đặc trưng hình học và màu sắc mạnh mẽ này mà không bị nhầm lẫn với phong nền phức tạp của môi trường tự nhiên như lá cây hay đất đá. - Các loài có kích thước nhỏ hoặc có xu hướng ẩn nấp, ngụy trang tốt như Cu Li Nhỏ (lông xám nâu tiếp màu cành cây) và Vuon Má Vàng (màu lông dễ lẫn vào bóng râm) đạt chỉ số AUC rất cao ở mức 0.98. Đây là kết quả xuất sắc trong điều kiện bẫy ảnh thực địa thường bị che khuất hoặc thiếu sáng. - Chỉ số AUC trung bình vĩ mô (Macro-average AUC) toàn cục đạt mức 0.99, khẳng định hệ thống có độ tin cậy cực kỳ cao, giảm thiểu tối đa tỷ lệ báo động giả (False Alarm) cũng như bỏ sót đối tượng (Missed Detection) ngoài thực tế dã ngoại.

Đường cong ROC biểu diễn mối quan hệ giữa tỷ lệ dương tính thật (True Positive Rate - TPR) và tỷ lệ dương tính giả (False Positive Rate - FPR) tại các ngưỡng quyết định

khác nhau. Chỉ số AUC đạt 0.99 chứng minh rằng mô hình có xác suất 99% xếp hạng một mẫu dương tính ngẫu nhiên cao hơn một mẫu âm tính ngẫu nhiên. Điều này khẳng định ranh giới phân loại mà ResNet-18 thiết lập trong không gian đặc trưng là vô cùng rõ ràng, giảm thiểu tối đa các sai số loại một (dương tính giả) và sai số loại hai (âm tính giả) cho toàn bộ 9 loài động vật hoang dã mục tiêu.

Phân tích sâu hơn về hiệu năng phân loại chi tiết theo từng loài cho thấy một số insight thú vị về khả năng trích xuất đặc trưng của mô hình: Các loài bị nhầm lẫn nhiều nhất là Hươu Sao và Cu Li Nhỏ, với 2 ảnh Hươu Sao bị phân loại nhầm thành loài khác. Nguyên nhân là do một số ảnh Hươu Sao chụp từ phía sau ở khoảng cách xa, khi đó các đốm trắng trên lưng trở nên khó phân biệt với họa tiết môi trường xung quanh. Ngược lại, Báo Gấm và Voọc Chà Vá đạt tỷ lệ nhận diện tuyệt đối 100% (Precision = Recall = 1.0) nhờ các đặc trưng hoa văn da và bộ lông vô cùng nổi bật, khó bị nhầm lẫn với bất kỳ loài nào khác trong tập 9 loài.

Về chỉ số Precision (độ chính xác dương tính) trung bình toàn cục, mô hình đạt mức 95.35%. Điều này có nghĩa là cứ 100 lần mô hình đưa ra nhận diện một loài cụ thể, thì 95.35 lần là chính xác. Chỉ số Recall (độ nhạy) trung bình đạt 95.10%, nghĩa là cứ 100 cá thể động vật thực sự xuất hiện trong ảnh, mô hình phát hiện thành công 95.10 cá thể. Chỉ số F1-Score (trung bình điều hòa của Precision và Recall) trung bình đạt 95.22%, xác nhận sự cân bằng giữa hai thước đo. Tất cả các chỉ số đánh giá đều vượt trội hơn đáng kể so với hai mô hình Baseline đối chứng, khẳng định tính hiệu quả của chiến lược Fine-tuning chọn lọc kết hợp Label Smoothing.

Ngoài ra, nhóm nghiên cứu cũng đã tính toán ma trận nhầm lẫn (Confusion Matrix) kích thước 9 x 9 trên tập Test để phân tích chi tiết các trường hợp phân loại sai. Kết quả cho thấy: đường chéo chính của ma trận chiếm ưu thế tuyệt đối (tổng $200/210 = 95.20\%$ ảnh nằm trên đường chéo), trong khi các phần tử ngoài đường chéo (các trường hợp nhầm lẫn) có giá trị rất nhỏ (0-2 ảnh mỗi ô). Không có bất kỳ cặp loài nào bị nhầm lẫn hệ thống (systematic confusion), chứng tỏ mô hình đã học được các đặc trưng phân biệt rõ ràng cho từng loài.

3. Đánh giá trên tập kiểm thử độc lập (Test Set) và So sánh với các mô hình đối chứng

Khi tiến hành đánh giá mô hình cuối cùng trên tập Test độc lập (hoàn toàn chỉ gồm ảnh gốc, không qua augment), hệ thống đạt độ chính xác toàn cục (Overall Accuracy) là 95.20% và độ nhạy trung bình (Average Recall) là 95.10%. Để chứng minh tính hiệu quả vượt trội của phương pháp đề xuất (Fine-tuning ResNet-18 kết hợp Label Smoothing), nhóm nghiên cứu đã xây dựng và huấn luyện hai mô hình đối chứng (Baseline) trên cùng một tập dữ liệu:

Bảng 5.1: Bảng so sánh hiệu năng giữa mô hình đề xuất và các mô hình đối chứng.

Mô hình	Phương pháp tiếp cận	Accuracy tập Test	Tình trạng Overfitting	Đặc tính hội tụ
Baseline 1 (CNN 3 lớp tự xây)	Huấn luyện từ đầu (From Scratch)	68.40%	Nghiêm trọng (sau Epoch 5)	Nhanh nhưng hiệu năng kém
Baseline 2 (ResNet-18)	Feature Extraction (Chỉ train FC)	84.60%	Thấp	Hội tụ chậm (đặc trưng chung)
Mô hình đề xuất	Fine-tuning Layer 3, 4 + Label Smoothing	95.20%	Không đáng kể	Tối ưu nhanh (15 Epochs)

- Mô hình Baseline 1 (CNN 3 lớp tự xây dựng): Được thiết kế đơn giản gồm 3 tầng tích chập kết hợp Max-Pooling và huấn luyện từ đầu (train từ scratch). Kết quả chỉ đạt độ chính xác tối đa 68.40% trên tập Test và bị hiện tượng quá khớp cực kỳ nghiêm trọng sau Epoch thứ 5. Nguyên nhân là do kích thước tập dữ liệu động vật hoang dã quá nhỏ, không đủ để mạng tích chập sâu tự học các bộ lọc đặc trưng chất lượng cao từ đầu. - Mô hình Baseline 2 (ResNet-18 Feature Extraction): Sử dụng kiến trúc ResNet-18 đã được tiền huấn luyện trên ImageNet, đóng băng hoàn toàn các tầng tích chập trích xuất đặc trưng và chỉ huấn luyện duy nhất lớp phân loại (Fully Connected

Layer) ở đầu ra. Mô hình này đạt độ chính xác 84.60% trên tập Test. Kết quả này khả quan hơn do tận dụng được các bộ lọc cơ bản (cạnh, góc, kết cấu) từ tập ImageNet, nhưng vẫn gặp giới hạn khi phân loại các đặc trưng chi tiết của các phân loài động vật cụ thể tại Việt Nam vốn không xuất hiện nhiều trong ImageNet. - Mô hình đề xuất (Fine-tuned ResNet-18): Rã đông và tinh chỉnh thêm Layer 3 và Layer 4, kết hợp kỹ thuật làm mịn nhãn (Label Smoothing = 0.1) giúp mô hình đạt độ chính xác vượt trội 95.20% (tăng 10.60% so với Feature Extraction và 26.80% so với CNN Custom). Sự chênh lệch đáng kể này chứng minh sự đúng đắn của chiến lược fine-tuning chọn lọc: giữ lại các tầng trích xuất đặc trưng bậc thấp (Layer 1, Layer 2) để giảm thiểu chi phí tính toán, đồng thời cho phép các tầng trích xuất đặc trưng bậc cao (Layer 3, Layer 4) học cách điều chỉnh bộ lọc để tập trung vào các chi tiết nhận dạng đặc thù của các loài động vật hoang dã quý hiếm trong Sách Đỏ Việt Nam.

Phân tích kết quả so sánh hiệu năng cho thấy sự vượt trội của mô hình đề xuất so với hai mô hình đối chứng. Mạng CNN 3 lớp tự xây dựng (Baseline 1) đạt kết quả thấp nhất (68.40%) do kiến trúc nông không đủ khả năng học các bộ lọc trích xuất đặc trưng phức tạp của động vật từ tập dữ liệu nhỏ. Mô hình ResNet-18 đóng băng hoàn toàn (Baseline 2) đạt kết quả khá hơn (84.60%) nhờ tri thức tiền huấn luyện ImageNet nhưng bị giới hạn do không được tinh chỉnh các lớp đặc trưng cấp cao cho phù hợp với dữ liệu thực tế. Mô hình đề xuất nhờ tinh chỉnh chọn lọc (Fine-tuned Layer 3 & 4) đã kết hợp được thế mạnh của cả hai phương pháp: tận dụng các đặc trưng cơ bản từ ImageNet và thích ứng sâu sắc với đặc trưng riêng biệt của động vật Sách Đỏ, mang lại độ chính xác vượt trội 95.20%.

LỜI KẾT

Dự án xây dựng hệ thống nhận diện 9 loài động vật hoang dã quý hiếm trong Sách Đỏ Việt Nam dựa trên kiến trúc ResNet-18 Transfer Learning tích hợp trực quan hóa Grad-CAM đã được hoàn thành trọn vẹn và đạt được tất cả các mục tiêu nghiên cứu ban đầu. Qua quá trình thực hiện đề tài, nhóm đã tích lũy được nhiều kinh nghiệm thực tiễn và kiểm chứng các lý thuyết học sâu một cách nghiêm túc.

1. Tổng kết các kết quả đạt được của dự án

Về mặt dữ liệu, nhóm nghiên cứu đã xây dựng thành công một pipeline xử lý dữ liệu chuẩn mực, sạch sẽ. Bằng cách phát hiện và khắc phục triệt để lỗi rò rỉ dữ liệu (Data Leakage) nghiêm trọng thông qua phương pháp chia tách tập huấn luyện, kiểm thử (Train/Val/Test) tỷ lệ 7:2:1 trước khi tiến hành tăng cường dữ liệu (Augmentation), mô hình đã đảm bảo tính khách quan và khả năng tổng quát hóa tối đa. Tập dữ liệu sau cân bằng đạt 210 ảnh/loài đối với tập Train giúp triệt tiêu hiện tượng lệch lớp gây thiên kiến dự đoán.

Về mặt mô hình học sâu, việc áp dụng phương pháp Transfer Learning tinh chỉnh (Fine-tuning) chọn lọc các lớp tích chập bậc cao (Layer 3 và Layer 4) của ResNet-18 kết hợp với hàm mất mát CrossEntropyLoss làm mịn nhãn (Label Smoothing = 0.1) và bộ tối ưu hóa Adam đã mang lại hiệu năng phân loại vượt trội. Mô hình đạt độ chính xác toàn cục 95.20% trên tập Test độc lập và chỉ số AUC trung bình đạt 0.99. Những con số này minh chứng cho tính đúng đắn của phương pháp tiếp cận và cấu hình mạng.

Về mặt ứng dụng thực tiễn, nhóm đã phát triển thành công hai sản phẩm hoàn chỉnh: ứng dụng Desktop chuyên nghiệp viết bằng CustomTkinter hỗ trợ suy luận ảnh/video cục bộ, hiển thị xác suất top 3 lớp dự đoán và bản đồ nhiệt Grad-CAM; kết hợp cùng ứng dụng Web Streamlit trực tuyến dễ tiếp cận. Sự hiện diện của mô hình giải thích AI (XAI) bằng Grad-CAM giúp chuyên gia sinh học kiểm chứng được cơ sở suy luận của mô hình, đảm bảo tính minh bạch và độ tin cậy khi triển khai thực địa.

2. Hạn chế của đề tài

Mặc dù đạt được những kết quả rất khả quan, hệ thống vẫn tồn tại một số điểm hạn chế cần được cải thiện trong tương lai: - Quy mô tập dữ liệu còn tương đối giới hạn về số

lượng ảnh gốc trên mỗi loài (trung bình 200 ảnh/loài). Điều này hạn chế khả năng nhận diện con vật dưới các góc chụp quá dị biệt hoặc bị che khuất nặng. - Chất lượng ảnh bẫy camera ngoài tự nhiên thường chịu ảnh hưởng lớn bởi yếu tố thời tiết, ánh sáng ban đêm (hồng ngoại trắng đen) và nhiễu chuyển động (motion blur). Mô hình hiện tại mới chỉ được huấn luyện chủ yếu trên ảnh màu ban ngày, dẫn đến hiệu năng nhận diện ảnh hồng ngoại ban đêm chưa thực sự tối ưu. - Bản đồ nhiệt Grad-CAM trích xuất từ Layer 4 của ResNet-18 có độ phân giải không gian thô (chỉ 7x7 pixel trước khi nội suy), tạo ra các quang cảnh bao quanh đối tượng thay vì bám khít chi tiết cơ thể động vật, làm giảm độ chính xác định vị các đặc trưng nhỏ như tai, mắt, hay đuôi.

3. Hướng phát triển và mở rộng nghiên cứu

Để nâng cao hơn nữa giá trị thực tiễn và tính khoa học của đề tài, nhóm định hướng các bước phát triển tiếp theo bao gồm: - Mở rộng tập dữ liệu cả về số lượng loài động vật hoang dã Việt Nam (bao gồm cả lớp thú, chim và bò sát quý hiếm) lẫn sự đa dạng của các bối cảnh bẫy ảnh thực địa (ảnh chụp đêm, ảnh thời tiết xấu). - Thử nghiệm và so sánh với các kiến trúc mạng mạnh mẽ hơn như ResNet-50, ConvNeXt hoặc Vision Transformers (ViT) để khai thác các đặc trưng không gian đa tỷ lệ và cơ chế chú ý toàn cục (self-attention). - Phát triển module phát hiện vật thể (Object Detection) dựa trên các kiến trúc như YOLOv8/v10 hoặc Faster R-CNN để không chỉ phân loại loài mà còn xác định vị trí tọa độ, đếm số lượng cá thể xuất hiện đồng thời trong một khung hình. - Tích hợp xử lý chuỗi thời gian (temporal context) trên các luồng video bẫy ảnh để giảm thiểu sai số nhận diện thông qua việc phân tích hành vi chuyển động liên tục của động vật hoang dã.

Sau quá trình nghiên cứu, triển khai thực nghiệm và đánh giá hệ thống, nhóm đã hoàn thành đầy đủ các mục tiêu đề ra cho đề tài "Xây dựng hệ thống nhận diện 9 loài động vật Sách Đỏ Việt Nam dựa trên kiến trúc ResNet-18 Transfer Learning tích hợp giải thích mô hình Grad-CAM". Các kết quả cụ thể đạt được bao gồm:

1. Xây dựng tập dữ liệu thực tế: Thu thập thành công tập dữ liệu của 9 loài động vật hoang dã quý hiếm cần bảo tồn tại Việt Nam. Nhóm đã chia tập Train/Val/Test độc lập theo tỉ lệ 7:2:1 trước khi thực hiện tăng cường dữ liệu vật lý (file canbang.py). Cách

làm này giúp giải quyết lỗi rò rỉ dữ liệu (Data Leakage) và khắc phục sự mất cân bằng giữa các lớp dữ liệu bằng cách đưa số lượng ảnh tập Train về mức 210 ảnh mỗi loài.

2. Huấn luyện mô hình đạt hiệu năng tốt: Áp dụng phương pháp Transfer Learning và tinh chỉnh (Fine-tuning) hai khối tích chập cuối của mạng ResNet-18, kết hợp hàm mất mát CrossEntropyLoss có kỹ thuật làm mịn nhãn (Label Smoothing = 0.1) và bộ tối ưu Adam với tốc độ học phân cấp. Mô hình đề xuất đạt độ chính xác (Accuracy) 95.20% trên tập kiểm thử độc lập, cao hơn rõ rệt so với các mô hình Baseline đối chứng.

3. Tăng tính trực quan cho mô hình AI: Tích hợp thành công giải thuật Grad-CAM để biểu diễn vùng chú ý của mạng nơ-ron dưới dạng bản đồ nhiệt (heatmap). Sự kết hợp này giúp giải thích quá trình ra quyết định của mô hình AI, giúp cán bộ kiểm lâm hay nhà nghiên cứu dễ dàng đối chiếu đặc điểm nhận dạng bên ngoài của động vật với vùng tập trung phân tích của mạng nơ-ron.

4. Hoàn thiện ứng dụng thực tế: Phát triển thành công hai phiên bản phần mềm: Ứng dụng Desktop (CustomTkinter) sử dụng kỹ thuật đa luồng giúp giao diện hoạt động mượt mà khi xử lý video và tích hợp bộ lọc nhận diện động vật lạ (OOD Filter); cùng ứng dụng Web trực tuyến (Streamlit) gọn nhẹ, hỗ trợ hiển thị lưới ảnh tóm tắt kết quả phát hiện và áp dụng thuật toán bỏ qua khung hình (Frame Skipping) để tối ưu tài nguyên máy chủ.

Nhóm thực hiện đề tài xin gửi lời cảm ơn chân thành đến ThS. Huỳnh Thái Học. Sự định hướng chuyên môn và hướng dẫn tận tình của thầy trong suốt quá trình học tập và làm đồ án là yếu tố quyết định giúp nhóm hoàn thành tốt đề tài này.

TAI LIỆU THAM KHẢO

1. He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In Proceedings of the IEEE conference on computer vision and pattern recognition (pp. 770-778). DOI: 10.1109/CVPR.2016.90. (Tài liệu gốc về cấu trúc khối dư Residual Block và mạng ResNet).
2. Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., & Batra, D. (2017). Grad-cam: Visual explanations from deep networks via gradient-based localization. In Proceedings of the IEEE international conference on computer vision (pp. 618-626). DOI: 10.1109/ICCV.2017.74. (Đề xuất gốc về giải thích AI Grad-CAM).
3. Kingma, D. P., & Ba, J. (2014). Adam: A method for stochastic optimization. arXiv preprint arXiv:1412.6980. URL: <https://arxiv.org/abs/1412.6980>. (Công bố gốc thuật toán tối ưu học sâu Adam).
4. Szegedy, C., Vanhoucke, V., Ioffe, S., Shlens, J., & Wojna, Z. (2016). Rethinking the inception architecture for computer vision. In Proceedings of the IEEE conference on computer vision and pattern recognition (pp. 2818-2826). DOI: 10.1109/CVPR.2016.308. (Cơ sở toán học về Label Smoothing).
5. Deng, J., Dong, W., Socher, R., Li, L. J., Li, K., & Fei-Fei, L. (2009). Imagenet: A large-scale hierarchical image database. In 2009 IEEE conference on computer vision and pattern recognition (pp. 248-255). DOI: 10.1109/CVPR.2009.5206848. (Cơ sở dữ liệu trích xuất đặc trưng nền tảng).
6. Kaufman, S., Rosset, S., Perlich, C., & Stitelman, O. (2012). Leakage in data mining: Formulation, detection, and avoidance. ACM Transactions on Knowledge Discovery from Data (TKDD), 6(4), 1-21. DOI: 10.1145/2382577.2382579. (Lý thuyết cơ sở rò rỉ dữ liệu).
7. Bộ Khoa học và Công nghệ & Viện Khoa học và Công nghệ Việt Nam. (2007). Sách Đỏ Việt Nam - Phần I: Động vật. Nhà xuất bản Khoa học tự nhiên và Công nghệ. (Tài liệu phân loại và cảnh báo hiện trạng động vật Việt Nam).

8. Paszke, A., Gross, S., Massa, F., Lerer, A., & Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems* (pp. 8024-8035). URL: <https://papers.nips.cc/paper/9015>. (Thư viện Deep Learning PyTorch).